

UNIT



Introduction to Finite Automata, Non-deterministic and Deterministic Finite Automata

SYLLABUS

Introduction to Finite Automata: Structural Representations, Automata and Complexity, The Central Concepts of Automata Theory – Alphabets, Strings, Languages, Problems.

Nondeterministic Finite Automata: Formal Definition, An Application, Text Search, Finite Automata with Epsilon-Transitions.

Deterministic Finite Automata: Definition of DFA, How a DFA Process Strings, The Language of DFA, Conversion of NFA with ϵ -transitions to NFA without ϵ -transitions. Conversion of NFA to DFA.

LEARNING OBJECTIVES

- ✓ Definition of Finite Automata
- ✓ Types of Structural Representations
- ✓ Issues of Automata and Complexity
- ✓ Central Concepts of Automata Theory
- ✓ Formal Definitions and Significance of DFA and NFA
- ✓ Concept of Finite Automata with Epsilon Transitions
- ✓ Conversion of NFA with ϵ -transitions to NFA without ϵ -transitions
- ✓ Conversion of NFA to DFA.

INTRODUCTION

Automaton is defined as a self-operational system which does not require any explicit human involvement in performing its function. It plays a vital role in analyzing the boundaries of computation. Structural representations of finite automaton are used for representing the data in a systematic manner. A finite automaton (or) Finite state machine is a mathematical model that takes string (w) as its input and checks whether the string (w) belongs to the given language (L) or not. Finite automaton model is considered as an essential part of computer science. It is a five tuple machine consisting of various components like states (Q), input symbols (Σ), transition function (δ), start symbol (q_0) and final state (F). It is denoted by $M = (Q, \Sigma, \delta, q_0, F)$. In finite automaton, each states (Q) includes a transition (δ) defined on input symbols (Σ). A finite automaton is of two types i.e., Deterministic Finite Automaton (DFA) and Non-deterministic Finite Automaton (NFA/NFA).

A deterministic finite automaton defines only single transition from each state on any input symbol. Whereas, a non-deterministic finite automaton defines more than one transition from each state on the same input symbol. One of the distinguishing feature of NFA is that, it can also read null string (ϵ) as an input. A transition including ϵ is called as epsilon transition. Besides this, for every NFA there exists an equivalent DFA accepting the same language (L).

1.1 INTRODUCTION TO FINITE AUTOMATA

1.1.1 Structural Representations, Automata and Complexity

Q1. Define Finite Automaton. Explain about the model of Finite Automaton.

Answer :

Jan./Feb.-23(R18), Q2(a)

Finite Automata

A Finite Automata or Finite State Machine (FSM) is a mathematical model that takes string (w) as its input and checks whether it belongs to the given language or not. A finite state machine is mainly used in designing the text editors, language processors and lexical analyzers.

A finite automata (M) is designed using five tuples such as,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

Q = A finite set of states i.e., $\{q_1, q_2, q_3, \dots\}$

Σ = A finite set of input symbols

δ = A transition function that defines rule for change of state

q_0 = An initial state or start state where $q_0 \in Q$

F = The set of final state or accepting state $F \subseteq Q$

Model of Finite Automata

The model of finite automata is shown in figure below,

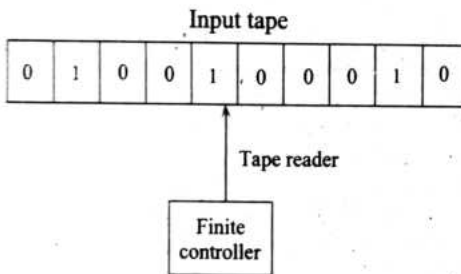


Figure: Model of Finite Automata

The model of finite automata consist of,

- (i) Input tape
 - (ii) Tape reader
 - (iii) Finite controller.
- (i) **Input Tape:** It is a sequential tape consisting of finite number of cells where each cell can hold only single input symbol.
 - (ii) **Tape Reader:** It performs the read operation cell-by-cell and reads only one input symbol (from each cell) at a given instance.
 - (iii) **Finite Controller:** It takes decision about which input symbol to read next after receiving a particular input

Q2. List the elements of finite automata.
Answer :

The elements of finite automata are,

1. State
2. Transition.

1. State

State defines the set of properties exchanged between the objects using communication channels. Any modification done in the property is detected by the receiving object.

Types of States

The different types of states are,

(a) Start State

It is the initial state of Finite State Machine (FSM).

(b) Accepting State

It is a state in which the string is accepted.

(c) Next State

It is the state which is immediately figured after the current state defined by the transition function.

(d) Universal State

It is the state, in which FSM accepts only those moves that leads to acceptance.

(e) Dead/Trap State

It is the state in which the input symbol terminates on itself.

2. Transition

Transition defines the change occurred when an object moves from one state to another. The different ways of representing transition are,

- (a) Transition diagram
- (b) Transition table
- (c) Transition function.

(a) Transition Diagram

A transition diagram describes the behavior of a finite automata. It consists of a collection of nodes or circles called states and directed line segments between each state called node edges. In finite automata, states are used for indicating the conditions that occur during the scanning process of the input. The scanning of input is usually done in order to search for an input that matches one of the several patterns. Moreover, the edges that carry transitions are used for indicating the next possible input that causes the automata to change the state.

The transition diagram which associates more than one input symbol to an edge is called non-deterministic transition diagram, whereas the transition diagram which associate unique input symbol on edge is called deterministic transition diagram.

Some conventions of transition diagrams are as follows.

1. It has one start state (or) initial state and one or more end states or final states or accepting states.
2. The states in transition diagram are represented using a circle (○).
3. The set of input symbols in transition diagram is associated to an edge i.e., $\xrightarrow{0}$ (Here 0 is an input symbol).
4. The start state is indicated by an edge labelled "start" $\xrightarrow{\text{Start}}$ while the end state or accepting or final state is indicated by a double circle (⊙).
5. A transition diagram always begins in the start state for reading input and ends in the final state indicating that a string (w) is found.

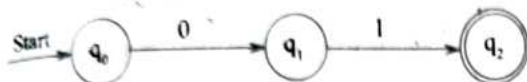


Figure (1): Transition Diagram

(b) Transition Table

It is the tabular representation of a given finite automata. It consists of rows and columns, wherein rows correspond to the input state and the column corresponds to the input symbol of given input state. The transition table is used to define a transition function δ .

Example

Consider the finite automata,

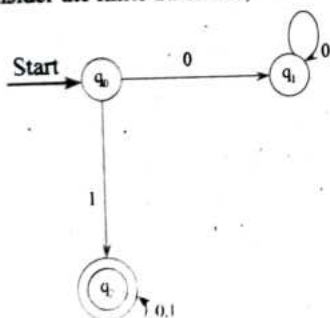


Figure (2): Transition Diagram

The transition table for the above finite automata is.

Input States	Input Symbols	
	0	1
$\rightarrow q_0$	q_1	q_2
q_1	q_1	ϕ
$\odot q_2$	q_2	q_2

Table: Transition Table

(c) Transition Function

A transition function defines the rules for change of the state. It takes two arguments a state and the input symbol. It returns a subset of output states i.e., more than one output state.

The properties of transition function are as follows.

Property 1

For any transition function ' δ ' and for any input symbol 'e'

$$\delta(q, \epsilon) = q$$

Explanation

The state of the transition system is changed only by an input symbol.

Property 2

For any transition function ' δ ', containing an input string s and input symbol b .

$$\delta(q, bs) = \delta(\delta(q, b), s)$$

and

$$\delta(q, sb) = \delta(\delta(q, s), b)$$

Explanation

The transition $\delta(q, bs)$ returns the next state after the first symbol of the string bs is read by the automaton.

The transition $\delta(q, sb)$ returns the state after the prefix of the string sb is read by the automaton.

Q3. Briefly explain the structural representations of finite automata.

Answer :

Structural Representations

Structural representations of finite automata are used for representing the data in a systematic manner. In automata theory, two important notations are used for structural representation. These notations are mostly used in software designing. The two types of structural representations are as follows.

1. Grammar
2. Regular expression.

1. Grammar

A grammar defines constructs of programming language. This form of representation is needed for those software that follow recursive structure for data processing. For instance, a parser that uses the recursively nested aspect of programming languages including arithmetic expressions, conditional expressions etc.

Example

Consider the grammar $\text{exp} \Rightarrow \text{exp} + \text{exp}$. This means that addition of two expressions using plus operator can result in a new expression.

Context-free grammar is an example of this form of representation.

2. Regular Expression

A regular expression is a concise notation for denoting regular set. It describes the language accepted by a finite automata. This form of representation is similar to that of finite automata, but considerably different from Grammars.

Examples

Consider the pattern "capitalized word accompanied by a space and two uppercase letters" UNIX-supported regular expression represent this pattern as

$[A-Z][a-Z]^{\wedge} [A-Z][A-Z]^{\wedge}$

Where

$[A-Z]$: Indicates all the upper case characters from A to Z

$[]$: Indicates space

$^{\wedge}$: Indicates any number of characters of the previous expression.

Q4. Explain in detail about automata and complexity.

Answer :

Automata and Complexity

Automata plays a vital role in analyzing the boundaries of computation. It involves two issues which are as follows,

1. Desirability and decidability
2. Intractability and tractability.

1. Desirability and Decidability

- (a) **Desirability:** Desirability is the study regarding all the functionality of the computer. More precisely, it is a study about what a computer can do, how it performs its task etc.
- (b) **Decidability:** Decidability refers to problem solving ability of the computer. A problem is said to be solvable or decidable if its supportive language is "recursive". That is, a solvable problem can be easily solved by using algorithmic approach. Here the word "decidable" refers to the word "recursive". Hence, it can be said that for a given set of problems if there exists an algorithm which can successfully produce two outputs namely "Yes" (or) "No", such set of problems are said to be set of decidable problems.

2. Intractability and Tractability

- (a) **Intractability:** Intractability is the study regarding the efficient working of a computer.
- (b) **Tractability:** Tractability describes the problem solving ability of the computer within the time specified by the polynomial functions (i.e., slowly growing functions). A problem is said to be tractable if the time and space complexity is reasonable to solve a problem.

1.1.2 The Central Concepts of Automata Theory - Alphabets, Strings, Languages, Problems

Q5. What is Automata? List the applications of finite automata. Discuss why to study the concept of automata.

OR

Describe in brief about applications of Finite Automata.

Jan./Feb.-23(R18), Q3(a)

(Refer Only Topic: Applications of Finite Automata)

OR

Write down the applications of finite automata.

(Refer Only Topic: Applications of Finite Automata)

Answer : (Dec.-18(R16), Q1(b) | April-18(R16), Q1(b))

Automata

Automaton is defined as a self-operational system which doesn't require any explicit human involvement in performing its function. The basic function carried out by automaton include,

- (i) Generating information
- (ii) Transforming information
- (iii) Transmitting information.

Applications of Finite Automata

The applications of finite automata are as follows,

1. It is used by language processors and compilers inorder to carry out the process of tokenization. In this process, the program is broken down into sequence of tokens like identifiers, constants and keywords.
2. It is used in deleting/eliminating comments and white spaces from a program code.
3. It is used by text processors inorder to search for a particular string in a text file that matches a particular pattern.
4. It is used in various signal and speech processing systems to transform incoming signals.
5. It is used in various industrial processing and in video game systems.

Study of Automata

Finite automata model is considered as the most essential part of computer science. It is used for various types of hardware and software, and must be studied for following reasons.

- (i) It helps in designing and checking the behaviour of digital circuits within the software.
- (ii) It helps the lexical analyzer phase to convert the input text to logical unit.

(iii) It allows the software to scan large quantity of text including collection of web pages. Also, with automata the user can detect the repeated words phases or pattern within the text.

(iv) It allows the software to verify various types of systems which have specific finite number of distinct states that includes communication protocols or protocols that are used for exchange of information.

Q6. Define the following terms,

- Alphabet
- String
- Language
- Problem
- Powerset.

Answer :

(a) **Alphabet**

An alphabet or character is any non-empty finite set of symbols. Letters and characters are examples of symbols. An alphabet is represented with the symbol Σ . Some example of alphabet are,

- Binary alphabet, $\Sigma = \{0, 1\}$
- Set of all upper-case letter,
 $\Sigma = \{A, B, \dots, Z\}$

(b) **String**

A finite sequence of symbols drawn from an alphabet is known as string 'S'. For example, if the alphabet, $\Sigma = \{0, 1\}$ then string S drawn from this alphabet is 001100.

A string has certain length that specifies the number of symbols it contains. The length of a string S denoted as $|S|$. The string 001100 is a string of length six i.e., $|001100| = 6$. The string of length zero is called as an empty string and is denoted by the symbol ϵ .

(c) **Language**

The language denoted by L is a set of strings over the alphabet Σ . For example, English is a language where the collection of English words is a set of strings over the alphabet consisting of all the letters. Similarly, any programming language like C is a language where the programs are a subset of strings formed over the alphabet of the language. However these languages are difficult to specify. Moreover the languages like ϕ , or empty set are also languages. Some examples of languages are as follows,

- ❖ The language of all strings consisting of alternate 0's and 1's : $\{\epsilon, 01, 010, 01010, \dots\}$
- ❖ The set of strings consisting of equal number of 0's and 1's : $\{\epsilon, 01, 0011, 101001, \dots\}$
- ❖ The empty language over any alphabet : $\{\phi\}$
- ❖ For any Σ, Σ^* is a language.

(d) **Problem**

A problem can be defined as a question which is useful to determine whether a string belongs to some specific language. In other words, if L is a language over an alphabet S the problem is,

$$L = \{W \in \Sigma^* / \text{Only if } W \in L\}.$$

(e) **Powerset**

A set 'A' is said to be the power set of set 'B' if it contains all the elements of set 'B'.

Example

If $A = \{a, b, c\}$ then

$$B = \{\phi, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}, \{a, c\}, \{a, b, c\}\}.$$

The number of elements in a power set 'B' will be equal to 2^n where n is the number of elements in set 'A'.

Q7. What are different operations performed on strings and languages? Explain with example.

Answer :

Operations on Languages

The operations on languages are called the regular operations. These are as follows,

- Union
- Concatenation
- Closure or Kleene star
- Positive Closure.

1. **Union**

Union of two languages L_1 and L_2 , denoted as $L_1 \cup L_2$ contain all strings (w) from both the languages.
 $L_1 \cup L_2 = \{w \mid w \text{ is in } L_1 \text{ or } w \text{ is in } L_2\}.$

Examples

- Let, $L_1 = \{0, 01, 10, 11\}$ and
 $L_2 = \{0, 01, 001\}$
 $L_1 \cup L_2 = \{0, 01, 001, 10, 11\}$
- Let, $L_1 = \{0, 00, 000, \dots\}$ and
 $L_2 = \{0, 000, 0000, \dots\}$
 $L_1 \cup L_2 = \{0\}^*$

2. **Concatenation**

Concatenation of two languages L_1 and L_2 , denoted as $L_1 L_2$ or $L_1 \circ L_2$ contains the strings from L_1 followed by strings from L_2 .
 $L_1 L_2 = \{st \mid w_1 \text{ in } L_1 \text{ and } w_2 \text{ is in } L_2\}.$

Examples

- Let, $L_1 = \{a, ab, ba, b\}$ and $L_2 = \{b\}$
 $L_1 L_2 = \{ab, abb, bab, bb\}$
- Let, L be a language. Then $L\phi = \phi L = \phi$.

3. Closure or Kleene Star

Kleene star of a language L , denoted as L^* contains all the strings obtained by concatenating zero or more strings from L .

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Where, $L^i = L^{i-1} L$

Example

ϕ^* is the empty string.

Let $L = \{0, 1\}$. $L^* = \{\epsilon, 0, 1, 01, 10, 000, 001, \dots\}$.

4. Positive Closure

Positive Closure of a language L , denoted as L^+ contains all the strings obtained by concatenating one or more strings from L .

$$L^+ = \bigcup_{i=1}^{\infty} L^i$$

Example

Let $L = \{0, 1\}$ then L^+ is the set of all strings over the alphabet $\Sigma = \{0, 1\}$ having atleast one or more digits.

Operations on Strings

Various operations on strings are as follows,

1. Concatenation
2. Reverse
3. String exponentiation.

1. Concatenation

Concatenation of two strings a_1 and a_2 is a string w formed by writing the string a_1 followed by the string a_2 . It is written as $w = a_1 a_2$ and the length of w is $|w| = |a_1| + |a_2|$.

Example

If $a_1 = abc$ and $a_2 = def$ then $a_1 a_2 = abcdef$.

2. Reverse

The reverse of string ' w ' is formed by writing the symbols of ' w ' in backward direction. It is denoted by w^R .

Example

If $w = abcd$ then $w^R = dcba$.

3. String Exponentiation

String exponentiation is defined as $w^i = w^{i-1} w$, for $i > 0$.

When, $i = 0$, $w^0 = \epsilon$

where, ϵ represents empty string.

Example

If $w = abc$ then $w^2 = (abc)^2 = abcabc$.

1.2 NONDETERMINISTIC FINITE AUTOMATA

1.2.1 Formal Definition

Q8. Define NFA. Explain its significance and functions.

Answer :

Non-deterministic Finite Automata (NFA)

A Non-deterministic Finite Automata (M) is reverse of deterministic algorithm. It is a collection of five-tuple namely,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

Q = A finite set of states i.e., $\{q_0, q_1, q_2, \dots\}$

Σ = A finite set of input symbols including the input symbol ϵ

δ = A transition function defines the rules for change of the state. It takes two arguments a state and the input symbol and returns a subset of output states (more than one output state) i.e., $Q \times \Sigma \rightarrow 2^Q$

q_0 = An initial state or start state, $q_0 \in Q$

F = The set of final states or accepting states.

Example

The NFA recognizing the language having substring 001 over $\Sigma = \{0, 1\}$ is as follows,

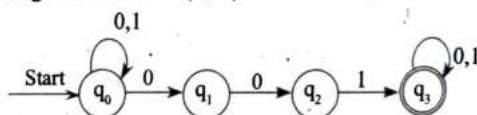


Figure: NFA

In the above transition graph, the NFA $M = (Q, \Sigma, \delta, q_0, F)$

Where,

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = q_0$$

$$F = q_3$$

Significance and Functions of NFA

An NFA is a machine to recognize a language. It reads a string of input symbols and determines whether it is acceptable or not. The machine starts in an arbitrary initial state and reads a string of input symbols from its alphabet. For each input symbol it transitions to a new state until it reads the complete input string. An NFA uses the state transition function δ to determine the next state. The function takes the current state and input symbol and returns the next state.

Unlike DFA, NFA is non-deterministic that is for any input string it can transit to some subset of all possible states. Another distinguishing feature of NFA is that it can also read null string (ϵ) as input. With null input symbol it transits to another state immediately without reading next input symbol. This can be repeated many times.

At any time the state of the machine is determined using the current input symbol as well as an arbitrary number of subsequent input events.

An NFA accepts a string if there is some sequence of possible moves on input string that put it in one of the accepting states. Otherwise, the string is rejected. The set of input strings accepted by an NFA forms the language of it. The language recognized by NFA is called regular language.

An NFA can be represented graphically by a directed graph called as a transition diagram. For every NFA there is an equivalent Deterministic Finite Automata (DFA) accepting the same language.

Q9. What is extended transition function of an NFA? Also discuss about the language of an NFA.

Answer :

Extended Transition Function of an NFA

The extended transition function of an NFA is defined as the function that can start at state q and accept a string from the input symbols S and reaches the set of states (say $n_1, n_2, n_3, \dots, n_m$). Its aspect is similar to the extended transition function of DFA. It is also denoted as $\hat{\delta}$.

It is defined in the formal way as,

Consider the following,

$S = az$ (string of input symbols)

Where,

a is all the symbols of S excluding last symbol

z is last symbol of S

$\hat{\delta}(q_1, a) = \{m_1, m_2, m_3, \dots, m_k\}$

$m_1, m_2, \dots, m_k$ are the states reached upon processing the string ' a '

Let $\bigcup_{i=1}^k \delta(p_i, 3) = \{n_1, n_2, n_3, \dots, n_m\}$

Where, $n_1, n_2, n_3, \dots, n_m$ are the states reached upon processing the string z .

Then,

$\hat{\delta}(q, S) = \{n_1, n_2, n_3, \dots, n_m\}$

The extended transition function $\hat{\delta}(q, \epsilon) = \{q\}$ which means, the state will remain same if the input has not read.

Example

Consider the NFA for the Language

$L = \{S \mid S \text{ is a set of all strings with 1 as second last symbol}\}$

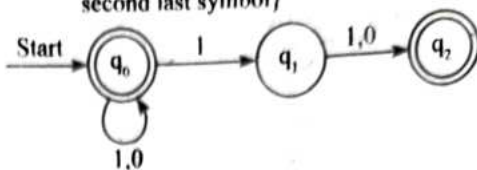


Figure: NFA for the above Language L

Now, consider the string that should be accepted by the NFA using extended transition function $\hat{\delta}$.

Let $S = 01010$, then $a = 0101$ and $z = 0$

$$\hat{\delta}(q_0, \epsilon) = \{q_0\}$$

$$\hat{\delta}(q_0, 0) = \hat{\delta}(q_0, 0) = \{q_0\}$$

$$\hat{\delta}(q_0, 01) = \hat{\delta}(q_0, 1) = \{q_0, q_1\}$$

$$\begin{aligned} \hat{\delta}(q_0, 010) &= \delta(q_0, 0) \cup \delta(q_1, 0) \\ &= \{q_0\} \cup \{q_2\} \\ &= \{q_0, q_2\} \end{aligned}$$

$$\begin{aligned} \hat{\delta}(q_0, 0101) &= \delta(q_0, 1) \cup \delta(q_2, 1) \\ &= \{q_0, q_1\} \cup \phi \\ &= \{q_0, q_1\} \end{aligned}$$

$$\begin{aligned} \hat{\delta}(q_0, 01010) &= \delta(q_0, 0) \cup \delta(q_1, 0) \\ &= \{q_0\} \cup \{q_2\} \\ &= \{q_0, q_2\} \end{aligned}$$

Thus, the final states ' q_0 ', ' q_2 ' are reached.

Hence, $\hat{\delta}(q, S) = \{n_1, n_2, n_3, \dots, n_m\}$

Language of an NFA

The language of an NFA can be defined as all the strings accepted by NFA. The strings are accepted by NFA if the transitions made for processing the string contains at least one accepting state. It is defined in a formal way as,

Let L be a language and NFA be $N = \{Q, \Sigma, \delta, q_0, F\}$, then the language accepted by DFA denoted by $L(N)$ is given by,

$$L(N) = \{W \mid \hat{\delta}(q_0, W) \cap F \neq \emptyset\}$$

Q10. Design-NFA accepting string with a's and b's such that string containing two consecutive a's or two consecutive b's.

Answer :

Dec.-17(R13), Q3(b)

Let the NFA be $M = \{Q, \Sigma, \delta, q_0, F\}$

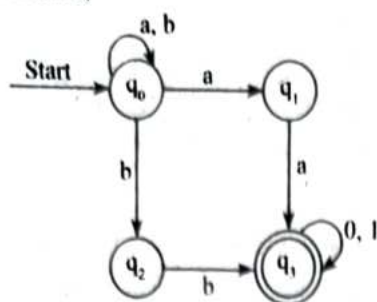
Where, $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$q_0 = q_0$

$F = q_3$

The NFA (transition diagram) that accepts set of strings containing two consecutive a's or two consecutive b's is as follows,



Q11. Design NFA to accept strings with a's and b's such that the string end with bb.

Answer :

May-13(R09), Q1(a)

Let the NFA be $M = \{Q, \Sigma, \delta, q_0, F\}$

Where,

Set of states (Q) = $\{A, B, C\}$

Set of input symbol (Σ) = $\{a, b\}$

The transition diagram (NFA) that accepts set of strings with 'a's and b's such that the string end with bb is as follows.

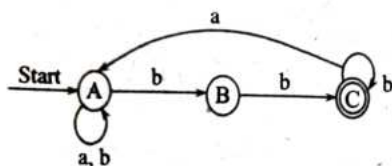


Figure: NFA

Here,

Initial state, (q_0) = A

Final state, (F) = C

The transition table (δ) is shown below,

State	Input Symbol	
	a	b
→ A	A	{A, B}
B	-	C
C	A	C

Table: Transition Table

Q12. Design a NFA accept the following strings over the alphabets $\{0, 1\}$. The set of all string that begins with 01 and ends with 11. Check for the validity of 01111 and 0110 strings.

Answer :

Given language $L = \{\text{Strings which begin with 01 and ends with 11}\}$

The given language (L) accepts strings like $\{0111, 01011, 01111\}$

The NFA of the above language is as follows,

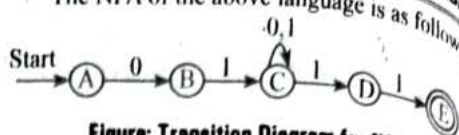


Figure: Transition Diagram for NFA

Let the above NFA be $M = (Q, \Sigma, \delta, q_0, F)$

Where, $Q = \{A, B, C, D, E\}$

$\Sigma = \{0, 1\}$

$q_0 = \{A\}$

$F = \{E\}$

The transition table is as follows,

States	Input Symbols	
	0	1
→ A	{B}	ϕ
B	ϕ	{C}
C	{C}	{C, D}
D	{ ϕ }	{E}
E	{ ϕ }	{ ϕ }

Table: Transition Table

(i) The processing of string $w = 01111$ is done as follows,

$$\delta(A, 0) = B$$

$$\delta(A, 1) = \phi$$

$$\delta(A, 01111) = \delta(\delta(A, 0), 1111)$$

$$= \delta(B, 1111)$$

$$= \delta(\delta(B, 1), 111)$$

$$= \delta(C, 111)$$

$$= \delta(\delta(C, 1), 11)$$

$$= \delta(D, 11)$$

$$= \delta(\delta(D, 1), 1)$$

$$= \delta(E, 1)$$

$$= E$$

$$\therefore \delta(A, 01111) = E \text{ (Final state)}$$

Since final state is reached, string $w = 01111$ is accepted by the above constructed NFA that accepts the language 'L'.

(ii) The processing of the given string $w = 0110$ is done as follows,

$$\delta(A, 0110) = \delta(\delta(A, 0), 110)$$

$$= \delta(B, 110)$$

$$= \delta(\delta(B, 1), 10)$$

$$= \delta(C, 10)$$

$$= \delta(\delta(C, 1), 0)$$

$$= \delta(D, 0)$$

$$= (C, D) \text{ (Non final state).}$$

Since, final state is not reached. The string $w = 0110$ is not accepted by the above constructed NFA that accepts the language L .

1.2.2 An Application, Text Search

Q13. How NFA is used for searching a text?

Answer :

NFA for Searching a Text

Non-deterministic finite automata are mostly used for performing text search from group of words usually called as keywords. To determine the occurrence of any keyword with in a text. For this reason a non-deterministic finite automata is designed. This automata is designed in such a way that it send signals upon seeing a keyword in the accepting state. Here, the document compressing text is sent from time to time to NFA so that it finds occurrences of the keyword in the text.

The NFA to search a text (word) is designed as follows.

- (i) The finite automata consists of a start state with a transition to every input symbol. For instance, all the ASCII characters which are printable. The start state shows the possible number of words after seeing some letters, number of guessed words etc.
- (ii) There exists K states for the keyword $m_1, m_2, \dots, m_k$. A transition from the start state of q_1 on symbol m_1 will exist wherein a transition from q_1 to q_2 exist on symbol m_2 and on forth q_k state represents that it is an accepting state and signifies that it has found all the keywords $m_1, m_2, \dots, m_k$.

1.2.3 Finite Automata with Epsilon-Transitions

Q14. Briefly discuss about Finite Automata with Epsilon-transitions. Dec.-18(R16), Q3(b)

OR

Explain about ϵ -closure (). Dec.-17(R13), Q1(b)

OR

Construct NFA with ϵ which accepts a language consisting the strings of any number of 0's followed by and number of 1's followed by any number 2's.

(Refer Only Topic: Example)

Answer :

May-17(R13), Q2(a)

Epsilon Closure (ϵ -moves)

Epsilon closure of state q , represented as ϵ -closure (or) ϵ -closure (q), is a set of NFA states including q , that are reachable from q on input symbol ϵ alone. ϵ -closure (q) cannot be an empty set because q is always reachable from itself irrespective of an input symbol. Therefore, ϵ -closure(q) always includes q .

If P is a set of states then.

$$\epsilon\text{-closure}(P) = \bigcup_{q \in P} \epsilon\text{-closure}(q).$$

NFA with ϵ -transitions

A Non-Deterministic Finite Automata, (NFA) with ϵ -transitions, M is defined by five tuples.

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

Q = A finite set of states i.e., $q_0, q_1, q_2, \dots$

Σ = A finite set of input symbols.

δ = A transition function that defines the rules for change of the state. It maps from $Q \times (\Sigma \cup \epsilon)$ to 2^Q .

q_0 = Initial state, $q_0 \in Q$.

F = The set of final state, $F \subseteq Q$.

Languages Accepted by Finite Automata with ϵ -transitions

An NFA with ϵ transition defined by $M = (Q, \Sigma, \delta, q_0, F)$ accepts set of strings only if it reaches the final state (q_f). That is, any string that starts from initial state (q_0) and ends at final state (q_f) is said to be acceptable by finite automata.

Let L be a language and w be the string of language, then the acceptance of language by NFA is given by,

$L(M) = \{w | w \in \Sigma^* \text{ and transition } (\delta) \text{ on } w \text{ starts from } q_0 \text{ and ends at } q_f\}$.

Example

Consider the below NFA with ϵ -transition that accepts a language consisting the string of any number of 0's followed by any number of 1's. Followed by any number of 2's.

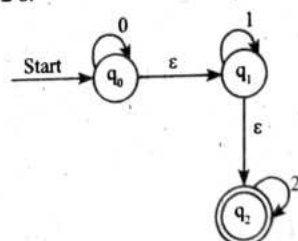


Figure: NFA with ϵ -Transitions

The transition table for above NFA is as follows,

States	Input symbols			
	0	1	2	ϵ
q_0	q_0	ϕ	ϕ	q_1
q_1	ϕ	q_1	ϕ	q_2
q_2	ϕ	ϕ	q_2	ϕ

Table: Transition Table

Let us check the acceptance of string $w = 00112$

$$\begin{aligned}
 \delta(q_0, 00112) &\vdash \delta(q_0, 0112) \\
 &\vdash \delta(q_0, 112) \\
 &\vdash \delta(q_0, \epsilon 112) \\
 &\vdash \delta(q_1, 112) \\
 &\vdash \delta(q_1, 12) \\
 &\vdash \delta(q_1, 2) \\
 &\vdash \delta(q_1, \epsilon 2) \\
 &\vdash \delta(q_2, 2) \\
 &\vdash \delta(q_2, \epsilon)
 \end{aligned}$$

Since final state is reached the string $w = 00112$ is accepted by finite automata.

Q15. Construct a Non-deterministic Finite Automata (NFA) with ϵ -transition(s) accepting set of all binary strings with 'n' number of 0's followed by 'm' number of 1's. Compute the ϵ -closure() for state in the NFA.

Answer :

The NFA with ϵ -transition accepting set of all binary strings with 'n' number of 0's followed by 'm' number of 1's is shown below,

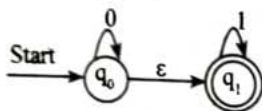


Figure: Transition Diagram for NFA- ϵ

For the above NFA- ϵ , ϵ -closure can be computed as follows,

$$\begin{aligned}
 \epsilon\text{-closure}(q_0) &= \{q_0, q_1\} \\
 \epsilon\text{-closure}(q_1) &= \{q_1\} \\
 \delta'(\{q_0\}, 0) &= \epsilon\text{-closure}(\delta(\{q_0\}, 0)) \\
 &= \epsilon\text{-closure}(q_0) \\
 &= \{q_0, q_1\} \quad \dots \text{New state} \\
 \delta'(\{q_0\}, 1) &= \epsilon\text{-closure}(\delta(\{q_0\}, 1)) \\
 &= \epsilon\text{-closure}(\phi) \\
 &= \phi \quad \dots \text{Empty state} \\
 \delta'(\{q_0, q_1\}, 0) &= \epsilon\text{-closure}(\delta(\{q_0, q_1\}, 0)) \\
 &= \epsilon\text{-closure}(\delta(q_0, 0) \cup \delta(q_1, 0)) \\
 &= \epsilon\text{-closure}(q_0 \cup \phi) \\
 &= \epsilon\text{-closure}(q_0) \\
 &= \{q_0, q_1\} \quad \dots \text{Old state} \\
 \delta'(\{q_0, q_1\}, 1) &= \epsilon\text{-closure}(\delta(\{q_0, q_1\}, 1)) \\
 &= \epsilon\text{-closure}(\delta(q_0, 1) \cup \delta(q_1, 1)) \\
 &= \epsilon\text{-closure}(\phi \cup q_1) \\
 &= \epsilon\text{-closure}(q_1) \\
 &= \{q_1\} \quad \dots \text{New state}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(\{q_1\}, 0) &= \epsilon\text{-closure}(\delta(\{q_1\}, 0)) \\
 &= \epsilon\text{-closure}(\phi) \\
 &= \phi \quad \dots \text{Empty state} \\
 \delta'(\{q_1\}, 1) &= \epsilon\text{-closure}(\delta(\{q_1\}, 1)) \\
 &= \epsilon\text{-closure}(q_1) \\
 &= \{q_1\} \quad \dots \text{Old state}
 \end{aligned}$$

Since, there are no new states, the process of computing ϵ -closure terminates.

Q16. Explain the uses of ϵ -transitions with an example.

Answer :

Uses of ϵ -transitions

The uses of ϵ -transitions are,

- It enables NFA to perform state transition even without the input string.
- It forms another reason for NFA to undergo number of states at a time.

Example

Consider the Languages $L = \{S \mid S \text{ is a set of all strings formed by any combination of } x, y, z\}$. The DFA for this Language would be,



Figure: DFA for the above Language

The above DFA does not provide clear idea about all the combinations of x, y, z . Thus, in this case ϵ -transitions of NFA will be very helpful as shown in the figure below.

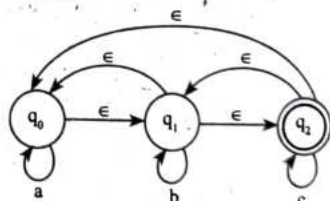


Figure: NFA with ϵ -transitions

The above NFA clearly indicates that, any combinations of x, y, z can be accepted by it. The ϵ -transition enables transition to various states without any input. For instance, for the letter 'c' appearing just beside 'a', it is possible to move from state q_0 to state q_2 .

Q17. Discuss about extended transition and languages for ϵ -NFA's.

Answer :

Extended Transitions and Languages Accepted by ϵ -NFA's

The extended transitions and languages accepted by ϵ -NFA's actually means string acceptance by ϵ -NFA through extended transition function. More precisely, it means extended transition function for ϵ -NFA's.

The extended transition function for ϵ -NFA's is defined as follows.

$$\hat{\delta}(q, \epsilon) = \epsilon \text{CLOSE}(q)$$

This equation states that when there is a path named ' ϵ ' then the transitions are allowed for state ' q ' from the all the ϵ -named paths.

Consider the string $S = az$.

Where,

a = all the symbols of S except the last symbol

z = last symbol of S .

$$\text{Let } \hat{\delta}(q, a) = \{m_1, m_2, m_3, \dots, m_k\}$$

The m_i 's are the states reached for processing the string ' a ' from state ' q '. These states may include one or more ϵ -transitions and may also possess other ϵ -transitions.

$$\text{Let } \bigcup_{i=1}^k \hat{\delta}(m_i, z) = \{n_1, n_2, n_3, \dots, n_m\}$$

The n_j 's are the set of states reached from state q upon processing string S .

$$\text{Then, } \hat{\delta}(q, a) = \bigcup_{j=1}^m \epsilon \text{CLOSE}(n_j)$$

The n_j 's here, are the set of states reached for processing the string S including ϵ -transitions that can be made after processing the last symbol z .

Thus, the extended transition function for ϵ -NFA's is $\hat{\delta}(q, S) = \bigcup_{j=1}^m \epsilon \text{CLOSE}(n_j)$.

1.3 DETERMINISTIC FINITE AUTOMATA

1.3.1 Definition of DFA, How a DFA Process Strings, The Language of DFA

Q18. Define Deterministic Finite Automata (DFA). Explain about the significance of DFA.

Answer :

Deterministic Finite Automata (DFA)

A deterministic finite automata (M) is a collection of five tuple, namely,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

Q = A finite set of state i.e., $Q = \{q_0, q_1, \dots, q_n\}$

Σ = A finite set of input symbol excluding ϵ

δ = A transition function that defines the rule for change of the states. It takes two arguments a state and an input symbol and returns a single output state i.e., $Q \times \Sigma \rightarrow Q$

q_0 = An initial state or start state

F = The set of final states or accepting states.

Significance of DFA

A deterministic finite automata is a finite machine that recognizes a given language.

It is represented diagrammatically by a labelled directed graph called a transition graph or transition diagram. The nodes of a graph represent the states and the edges represents the transition function. The initial state is indicated by an arrow head pointing to it. The final states are indicated by double circles.

Example

The DFA recognizing the language L = Set of all strings containing block of five consecutive 1's and atleast two 0's is shown in below figure.

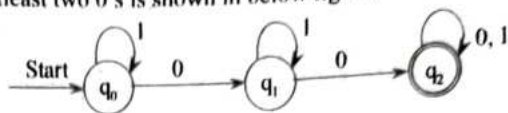


Figure: DFA

In the above transition graph, DFA is $M = (Q, \Sigma, \delta, q_0, F)$

Where,

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = q_0$$

$$F = q_2$$

Here, q_0 is an initial state and q_2 is the final state. A DFA in initial state reads a string of input symbols from its alphabet. For each input symbol it transitions to a new state until it reaches at the end of the string. In DFA there is a transition from each state on each input symbol. At any time the state of the machine is determined using its current input symbol.

A DFA accepts a string if the end of input string is in one of the final states. Otherwise the string is rejected.

It is said to be deterministic because there exists only one transition from a state for a given input symbol.

Q19. Explain how DFA processes strings with an example.

Answer :

A deterministic finite automata $M = (Q, \Sigma, \delta, q_0, F)$ accepts those set of strings (W) that reaches the final state (F). The strings which are accepted by DFA are called as language of DFA.

If $x_1, x_2, x_3, \dots, x_n$ are sequence of input symbols, then processing of these input symbol is done by considering a transition function of form.

$$\delta(q_0, x) = q_1$$

$$\delta(q_1, x) = q_2$$

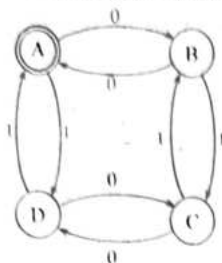
:

$$\delta(q_{i-1}, x) = q_i$$

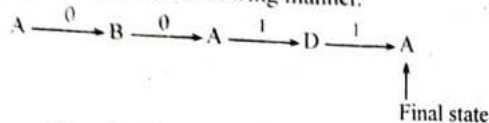
All the input symbols are processed sequentially called final state start till end. If the last input symbol (say x) reaches a final state F then it can be said that the set of strings on input $a_1, a_2, a_3, \dots, a_n$ is accepted by DFA (M).

Example

Consider the following finite automata.



Consider a string $W = 0011$. The processing of string is done in the following manner.



Since the final state is reached, it can be said that string 0011 is accepted by DFA and the string belongs to language of DFA.

Q20. What is extended transition function of DFA? Discuss about the languages accepted by a DFA.

Answer :

Extended Transition Function of DFA

Extended transition function of DFA can be defined as a function which can start at state ' q ' and accept a string S and finally reaches a state Q . In other words, the transition function is expanded to deal with words inspite of strings. Moreover it also enables the DFA to start at any state and accept any sequence of inputs. It is denoted by $\hat{\delta}$.

The extended transition function with respect to the length of the input string is defined as follows,

$$\hat{\delta}(q, S) = \delta(\hat{\delta}(q, a)z)$$

Where,

$q \rightarrow$ any state

$S \rightarrow$ a string of the form az

$a \rightarrow$ a string with all the symbols of string S except the last symbol.

$z \rightarrow$ Last symbol of strings.

$\hat{\delta}(q, a) \rightarrow$ a state undergone by the automaton processes all the symbols (i.e., a) except the last symbol (z) from S .

$\hat{\delta}(q, \epsilon) \rightarrow$ Indicates that the input has not read by the automaton and thus the automaton remains in same state, i.e. $\hat{\delta}(q, \epsilon) = q$.

Example

Consider the DFA for the Language,

$L = \{S | S \text{ is of even length and start with } 01\}$

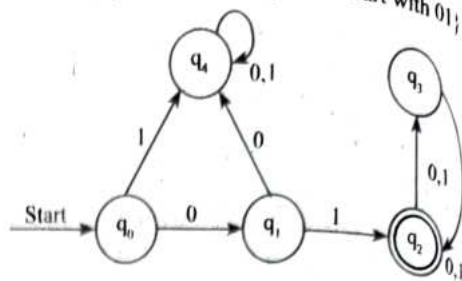


Figure: DFA for the above Language L

Languages Accepted by a DFA

Let L be a language and DFA be $M = \{Q, \Sigma, \delta, q_0, F\}$. The languages accepted by DFA denoted by $L(M)$ is given by,

$$L(M) = \{\omega | \hat{\delta}(q_0, \omega) \text{ reaches the final state } M\}$$

The DFA of $M = \{Q, \Sigma, \delta, q_0, F\}$ accepts a set of language (L) if a string (W) starts from initial state and reaches the final state/accepting state at the end.

Example

Consider the language $L = \{(a, b) | \text{where number of } a\text{'s are greater than } b\text{'s}\}$. The deterministic finite automata for the language is,

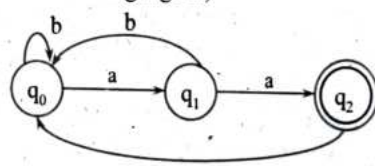
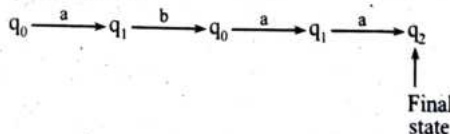


Figure: DFA

To check whether the string (W) belongs to given language or not consider a string $W = abaa$.

The acceptance of string is checked in the following manner.



Since final state is reached, it can be said that string W is accepted by language L .

Q21. Construct a DFA to accept the binary strings divisible by 5. Sep.-21(R18), Q2(a)

OR

Design a DFA which accepts set of all strings which are divisible by 5 for binary alphabet.

Answer :

Dec.-19(R16), Q2

The DFA accepting the set of strings divisible by 5 for binary alphabet (0, 1) is as follows,

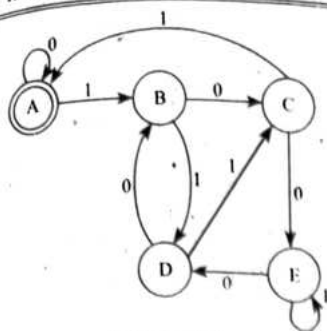


Figure: DFA

Let the above DFA be $M = \{Q, \Sigma, \delta, q_0, F\}$

Where,

$$Q = \{A, B, C, D, E\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = \{A\}$$

$$F = \{A\}$$

The transition table (δ) is,

State	Input Symbol	
	0	1
→ A	A	B
B	C	D
C	E	A
D	B	C
E	D	E

The given DFA accepts string like $\epsilon, 0, 000, 101$ or $1010, 0000, \dots$

Let us check whether the string 101 (i.e., 5) and 1010 (i.e., 10) is accepted by above DFA or not.

$$A \xrightarrow{1} B \xrightarrow{0} C \xrightarrow{1} A \text{ (Final state)}$$

Since final state is reached the string 101 (i.e., 5) is accepted by the above DFA.

Now let us check the acceptance of string 1010 (i.e., 10) by above DFA.

$$A \xrightarrow{1} B \xrightarrow{0} C \xrightarrow{1} A \xrightarrow{0} A$$

↑
Final state

Since final state is reached the string 1010 (i.e., 10) which is divisible by 5 is accepted by DFA.

The states in above DFA is divided in such a way that its state contain those strings that leave remainder 0 on division by 5.

Q22. Design DFA for the following over $\{a, b\}$.

- All strings containing not more than three a's.
- All strings that has atleast two occurrences of b between any two occurrence of a.

Answer : (Dec.-18(R16), Q2(b) | April-18(R16), Q3(a))

- All strings containing not more than three a's.

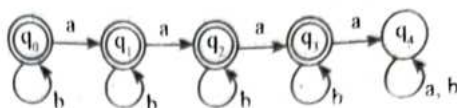


Figure: DFA for String Containing not more than Three a's

- All strings that has atleast two occurrences of b between any two occurrences of a.

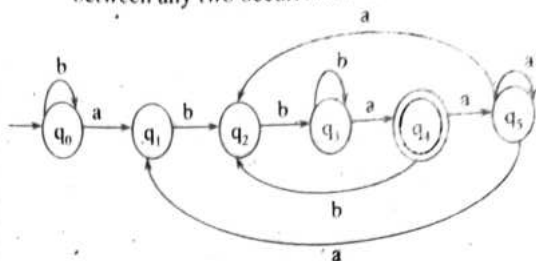


Figure: DFA

Q23. Construct a DFA accepting the set of strings ending with 00.

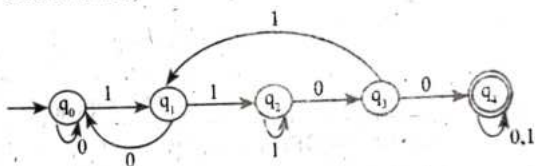
Answer :

April-18(R16), Q3(b)

Let the DFA be,

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

The DFA(M) that recognizes all possible strings over the alphabet $\{0, 1\}$ ending with two consecutive zero's will be,



Where,

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = q_0$$

$$F = q_4$$

The transition table (δ) will be,

States	Input Symbol	
	0	1
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_4	q_1
q_4	q_4	q_4

Table: Transition Table

Q24. Construct a DFA accepting the language $L = \{w : |w| \bmod 8 \neq 0\}$ on $\Sigma = \{a, b\}$.

Answer :

Given that,

$$L = \{w : |w| \bmod 8 \neq 0\} \text{ on } \Sigma = \{a, b\}$$

According to the language, the DFA must accept strings which is not divisible by 8.

Let the DFA be, $M = \{Q, \Sigma, \delta, q_0, F\}$

The DFA (M) will be,

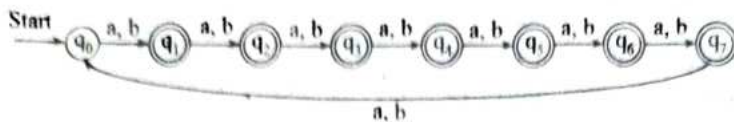


Figure: DFA

Where,

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = q_0$$

$$F = \{q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$$

δ = The transition table will be,

States	Input Symbol	
	a	b
q_0	q_1	q_1
q_1	q_2	q_2
q_2	q_3	q_3
q_3	q_4	q_4
q_4	q_5	q_5
q_5	q_6	q_6
q_6	q_7	q_7
q_7	q_1	q_1

Table: Transition Table

Q25. Construct a DFA to accept the binary strings consisting of even number of 0's and odd number of 1's.

Answer :

Sep.-21(R18), Q1(b)

Given that,

$$L = \{w : \{0, 1\}^* \mid w \text{ has even number of 0's and odd number of 1's}\}$$

The language for L is,

$$L(M) = \{010, 01011, 1, 001, 01011, 1010001, \dots\}$$

Let the DFA be $M = \{Q, \Sigma, \delta, q_0, F\}$

The DFA is,

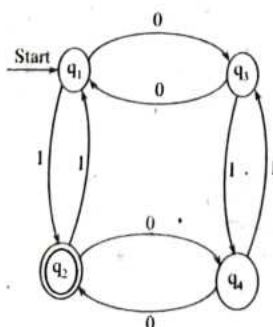


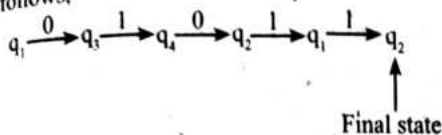
Figure: DFA

The transition table is,

States	Input Symbols	
	0	1
→ q_1	q_3	q_2
q_2	q_4	q_1
q_3	q_1	q_4
q_4	q_2	q_3

Table: Transition Table

Consider the string 01011 to be processed on DFA as follows,



Since, final state is reached, the string is accepted by the DFA. Hence the constructed DFA is correct.

Q26. Design a FA for the Language which accepts odd number of 0's and odd number of 1's over input alphabet $\Sigma = \{0, 1\}$.

Answer : Feb.-22(R18), Q1(a)

A finite automata which accepts all strings having odd number of 0's and 1's is shown below,

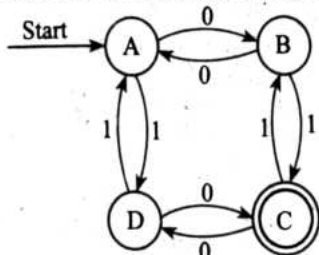


Figure: Finite Automata

Let, the finite automata be,

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

Where,

$$Q = \{A, B, C, D\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = \{A\}$$

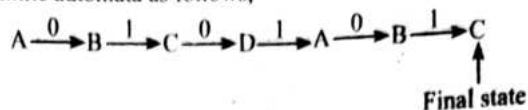
$$F = \{C\}$$

The transition table (δ) is,

States	Input Symbols	
	0	1
→ A	B	D
B	A	C
$\odot$ C	D	B
D	C	A

Table: Transition Table

The string 010101 can be processed on generated finite automata as follows,



As, final state is reached, the string is accepted by the finite automata. Hence, the constructed finite automata is correct.

1.3.2 Conversion of NFA with ϵ -transitions to NFA without ϵ -transitions

Q27: Explain the procedure of converting NFA with ϵ -transition to NFA without ϵ -transition with example.

Answer :

The procedure for converting NFA with ϵ -transitions (M) to NFA without ϵ -transitions consists of following steps,

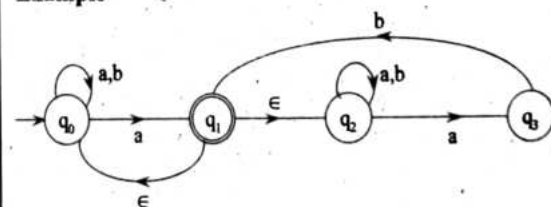
1. Find ϵ -closure(q) of all states of M . ϵ -closure(q) function returns a set of NFA states including q , that are reachable from q on input symbol ϵ alone.
2. Determine transition function as follows,

$$\delta'(q, \epsilon) = \epsilon\text{-closure}(q)$$

$$\delta'(q, a) = \epsilon\text{-closure}(\delta(\delta'(q, \epsilon), a))$$

$$= \epsilon\text{-closure}(q_p, a)$$
3. Final states of M' includes all states whose ϵ -closure contains a final state of M .

Example



ϵ -closure of all states is given as,

$$\epsilon\text{-closure}(q_0) = \{q_0\}$$

$$\epsilon\text{-closure}(q_1) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

$$\epsilon\text{-closure}(q_3) = \{q_3\}$$

Transition function is determined as follows,

$$\delta'(q_0, \epsilon) = \epsilon\text{-closure}(q_0) = \{q_0\}$$

$$\delta'(q_0, a) = \epsilon\text{-closure}(\delta(\delta'(q_0, \epsilon), a))$$

$$= \epsilon\text{-closure}(\delta(q_0, a))$$

$$= \epsilon\text{-closure}(q_0, q_1)$$

$$= \epsilon\text{-closure}(q_0) \cup \epsilon\text{-closure}(q_1)$$

$$= \{q_0\} \cup \{q_0, q_1, q_2\}$$

$$= \{q_0, q_1, q_2\}$$

$$\begin{aligned}
 \delta'(q_0, b) &= \epsilon\text{-closure}(\delta(\delta'(q_0, \epsilon), b)) \\
 &= \epsilon\text{-closure}(\delta(q_0, b)) \\
 &= \epsilon\text{-closure}(q_0) \\
 &= \{q_0\}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(q_1, a) &= \epsilon\text{-closure}(\delta(\delta'(q_1, \epsilon), a)) \\
 &= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, a)) \\
 &= \epsilon\text{-closure}(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)) \\
 &= \epsilon\text{-closure}(\{q_0, q_1\} \cup \phi \cup \{q_2, q_1\}) \\
 &= \epsilon\text{-closure}(\{q_0, q_1, q_2, q_1\}) \\
 &= \{q_0, q_1, q_2, q_1\}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(q_1, b) &= \epsilon\text{-closure}(\delta(\delta'(q_1, \epsilon), b)) \\
 &= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, b)) \\
 &= \epsilon\text{-closure}(q_0 \cup \phi \cup q_2) \\
 &= \{q_0, q_2\}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(q_2, a) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), a)) \\
 &= \epsilon\text{-closure}(\delta(q_2, a)) \\
 &= \epsilon\text{-closure}(q_2, q_1) \\
 &= \{q_2, q_1\}
 \end{aligned}$$

$$\begin{aligned}
 \delta'(q_2, b) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), b)) \\
 &= \epsilon\text{-closure}(\delta(q_2, b)) \\
 &= \epsilon\text{-closure}(q_2) \\
 &= \{q_2\}
 \end{aligned}$$

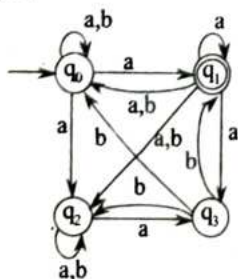
$$\begin{aligned}
 \delta'(q_3, a) &= \epsilon\text{-closure}(\delta(\delta'(q_3, \epsilon), a)) \\
 &= \epsilon\text{-closure}(\delta(q_3, a)) \\
 &= \epsilon\text{-closure}(\phi) \\
 &= \phi
 \end{aligned}$$

$$\begin{aligned}
 \delta'(q_3, b) &= \epsilon\text{-closure}(\delta(\delta'(q_3, \epsilon), b)) \\
 &= \epsilon\text{-closure}(\delta(q_3, b)) \\
 &= \epsilon\text{-closure}(q_1) \\
 &= \{q_0, q_1, q_2\}
 \end{aligned}$$

Transition table is,

q / Σ	a	b
$\rightarrow q_0$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
q_1	$\{q_0, q_1, q_2, q_1\}$	$\{q_0, q_2\}$
q_2	$\{q_2, q_1\}$	$\{q_2\}$
q_3	ϕ	$\{q_0, q_1, q_2\}$

Transition diagram of NFA without ϵ -transitions is shown in figure.



Figure

Q28. Construct NFA with ϵ which accepts a language consisting all strings of any number of 0's followed by any number of 1's followed by any number of 2's. And also convert into NFA without ϵ -transitions.

Answer :

May-19(R18), Q2

The NFA- ϵ for set of string accepting any number of 0's followed by any number of 1's followed by any number of 2's is,

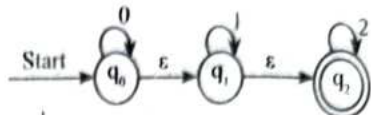


Figure (1): Transition Diagram for NFA- ϵ

Let the given NFA with ϵ -transitions,

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$\text{Where, } Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1, 2\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

Transition function is calculated as follows,

$$\delta'(q_0, \epsilon) = \epsilon\text{-closure}(q_0)$$

$$\delta'(q_0, 0) = \epsilon\text{-closure}(\delta(q_0, 0))$$

$$[\because \delta(q, a) = \epsilon\text{-closure}(\delta(\delta'(q, \epsilon), a))]$$

$$= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 0))$$

$$= \epsilon\text{-closure}(q_0)$$

$$= \{q_0, q_1, q_2\}$$

$$\delta'(q_0, 1) = \epsilon\text{-closure}(\delta(q_0, 1))$$

$$= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 1))$$

$$= \epsilon\text{-closure}(q_1)$$

$$= \{q_1, q_2\}$$

$$\delta'(q_0, 2) = \epsilon\text{-closure}(\delta(q_0, 2))$$

$$= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, 2))$$

$$= \epsilon\text{-closure}(q_2)$$

$$= \{q_2\}$$

$$\delta'(q_1, 0) = \epsilon\text{-closure}(\delta(q_1, 0))$$

$$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 0))$$

$$= \epsilon\text{-closure}(\phi)$$

$$= \{\phi\}$$

$$\delta'(q_1, 1) = \epsilon\text{-closure}(\delta(q_1, 1))$$

$$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 1))$$

$$= \epsilon\text{-closure}(q_1)$$

$$= \{q_1, q_2\}$$

$$\delta'(q_1, 2) = \epsilon\text{-closure}(\delta(q_1, 2))$$

$$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 2))$$

$$= \epsilon\text{-closure}(q_2)$$

$$= \{q_2\}$$

$$\begin{aligned}\delta'(q_2, 0) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), 0)) \\ &= \epsilon\text{-closure}(\delta(q_2, 0)) \\ &= \epsilon\text{-closure}(\phi) \\ &= \{\phi\}\end{aligned}$$

$$\begin{aligned}\delta'(q_2, 1) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), 1)) \\ &= \epsilon\text{-closure}(\delta(q_2, 1)) \\ &= \epsilon\text{-closure}(\phi) \\ &= \{\phi\}\end{aligned}$$

$$\begin{aligned}\delta'(q_2, 2) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), 2)) \\ &= \epsilon\text{-closure}(\delta(q_2, 2)) \\ &= \epsilon\text{-closure}(q_2) \\ &= \{q_2\}\end{aligned}$$

Transition table is,

States	Input Symbols		
	0	1	2
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	ϕ	$\{q_1, q_2\}$	$\{q_2\}$
q_2	ϕ	ϕ	$\{q_2\}$

Table: Transition Table

Transition diagram of NFA without ϵ -transitions is shown in the following figure,

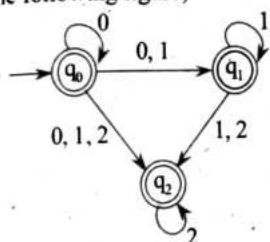


Figure (2): Transition Diagram of NFA

Processing the String 0112 on NFA with ϵ

$$\begin{aligned}\delta'(\epsilon\text{-closure}(q_0), 0) &= \delta'(\{q_0, q_1, q_2\}, 0) \\ &= \delta\{q_0, 0\} \cup \delta\{q_1, 0\} \cup \delta\{q_2, 0\} \\ &= \{q_0\} \cup \phi \cup \phi = \{q_0\}\end{aligned}$$

$$\begin{aligned}\delta'(\epsilon\text{-closure}(q_0), 01) &= \delta'(\epsilon\text{-closure}(\delta'(\epsilon\text{-closure}(q_0), 0)), 1) \\ &= \delta'(\epsilon\text{-closure}(\{q_0\}), 1) \\ &= \delta'(\{q_0, q_1, q_2\}, 1) \\ &= \delta\{q_0, 1\} \cup \delta\{q_1, 1\} \cup \delta\{q_2, 1\} \\ &= \phi \cup \{q_1\} \cup \phi = \{q_1\}\end{aligned}$$

$$\begin{aligned}\delta'(\epsilon\text{-closure}(q_0), 011) &= \delta'(\epsilon\text{-closure}(\delta'(\epsilon\text{-closure}(q_0), 01)), 1) \\ &= \delta'(\epsilon\text{-closure}(\{q_1\}), 1) \\ &= \delta'(\{q_1, q_2\}, 1) \\ &= \delta\{q_1, 1\} \cup \delta\{q_2, 1\} \\ &= \{q_1\} \cup \phi = \{q_1\}\end{aligned}$$

$$\begin{aligned}\delta'(\epsilon\text{-closure}(q_0), 0112) &= \delta'(\epsilon\text{-closure}(\delta'(\epsilon\text{-closure}(q_0), 011)), 2) \\ &= \delta'(\epsilon\text{-closure}(\{q_1\}), 2) \\ &= \delta'(\{q_1, q_2\}, 2) \\ &= \{q_1, 2\} \cup \delta\{q_2, 2\} \\ &= \phi \cup \{q_2\} = \{q_2\}\end{aligned}$$

Thus, the string 0112 is accepted by NFA with ' ϵ ' as final state is reached.

Processing the String 0112 on NFA

$$\delta'(q_0, \epsilon) = q_0$$

$$\begin{aligned}\delta'(q_0, 0) &= \delta(\delta'(q_0, \epsilon), 0) \\ &= \delta(\{q_0\}, 0) = \{q_0, q_1, q_2\}\end{aligned}$$

$$\begin{aligned}\delta'(q_0, 01) &= \delta(\delta'(q_0, 0), 1) \\ &= \delta(\{q_0, q_1, q_2\}, 1) = \{q_1, q_2\}\end{aligned}$$

$$\begin{aligned}\delta'(q_0, 011) &= \delta(\delta'(q_0, 01), 1) \\ &= \delta(\{q_1, q_2\}, 1) = \{q_1, q_2\}\end{aligned}$$

$$\begin{aligned}\delta'(q_0, 0112) &= \delta(\delta'(q_0, 011), 2) \\ &= \delta(\{q_1, q_2\}, 2) = \{q_2\}\end{aligned}$$

$\therefore$ Thus, the string 0112 is accepted by NFA as final state is reached.

Q29. Explain about the elimination of ϵ -transition with example.

Answer :

Elimination of ϵ -transition means conversion of ϵ -NFA to DFA or NFA.

Procedure for Converting NFA With ϵ -transition to DFA

The procedure uses two functions.

1. ϵ -closure(q)

This function returns a set of NFA states including q , that are reachable from the NFA state q on input symbol ϵ alone.

2. Move(T, a)

This function returns a set of NFA states that are reachable from the NFA state q in T on input symbol a . Here T itself consists of set of states and ' a ' is the input symbol.

The procedure consists of the following steps.

Step1: First, find the ϵ -closure of the start state (q_0) of NFA i.e., ϵ -closure(q_0). The resultant set of ϵ -closure(q_0) becomes the start state of the equivalent DFA.

Step2: Apply the ϵ -closure on the result of the function move(T, a) i.e., find ϵ -closure (move(T, a)).

Consider the resultant set as new single set.

Step3: If step 2 results in a new state, then repeat step 2 until there is no new state. The final states of a DFA are all those states having at least one accepting state of NFA.

Example

Consider the NFA with ϵ -transitions shown in the figure (1) which accepts the language $(a|b)^*abb$.

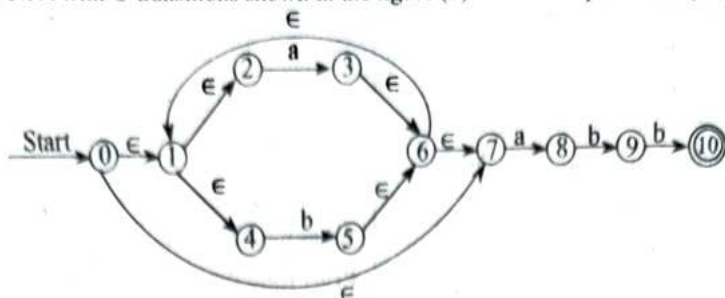


Figure (1): NFA Accepting $(a|b)^*abb$

In this example, initial state is '0' and final state is 10.

Step1: Find the ϵ -closure on the start state '0'.

$$\begin{aligned}\epsilon\text{-closure}(0) &= \{0, 1, 2, 4, 7\} \\ &= A\end{aligned}$$

Step2: Apply ϵ -closure on the result of the function move (A, a) .

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(A, a)) &= \epsilon\text{-closure}(\text{move}(\{0, 1, 2, 4, 7\}, a)) \\ &= \epsilon\text{-closure}(\{3, 8\}) \\ &= \{1, 2, 3, 4, 6, 7, 8\} \\ &= B\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(A, b)) &= \epsilon\text{-closure}(\text{move}(\{0, 1, 2, 4, 7\}, b)) \\ &= \epsilon\text{-closure}(\{5\}) \\ &= \{1, 2, 4, 5, 6, 7\} \\ &= C\end{aligned}$$

Step3: Repeat step 2 for two new states B and C.

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(B, a)) &= \epsilon\text{-closure}(\{3, 8\}) \\ &= \{1, 2, 3, 4, 6, 7, 8\} \\ &= B\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(B, b)) &= \epsilon\text{-closure}(\{5, 9\}) \\ &= \{1, 2, 4, 5, 6, 7, 9\} \\ &= D\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(C, a)) &= \epsilon\text{-closure}(\{3, 8\}) \\ &= B\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(C, b)) &= \epsilon\text{-closure}(\{5\}) \\ &= C\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(D, a)) &= \epsilon\text{-closure}(\{3, 8\}) \\ &= B\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(D, b)) &= \epsilon\text{-closure}(\{5, 10\}) \\ &= \{1, 2, 4, 5, 6, 7, 10\} \\ &= E\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(E, a)) &= \epsilon\text{-closure}(\{3, 8\}) \\ &= B\end{aligned}$$

$$\begin{aligned}\epsilon\text{-closure}(\text{move}(E, b)) &= \epsilon\text{-closure}(\{5\}) \\ &= C\end{aligned}$$

The new states of DFA are,

$$A = \{0, 1, 2, 3, 4, 7\}, \quad D = \{1, 2, 4, 5, 6, 7, 9\}$$

$$B = \{1, 2, 3, 4, 6, 7, 8\}, \quad E = \{1, 2, 4, 5, 6, 7, 10\}$$

$$C = \{1, 2, 4, 5, 6, 7\}$$

Transition Table

State	Input symbol	
	a	b
→A	B	C
B	B	D
C	B	C
D	B	E
*E	B	C

Equivalent DFA

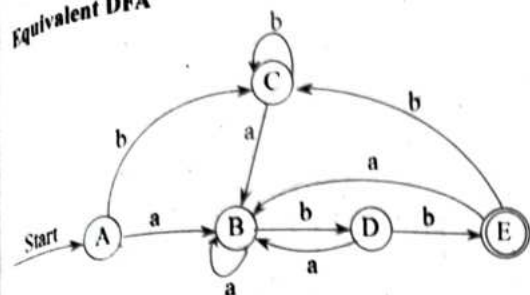
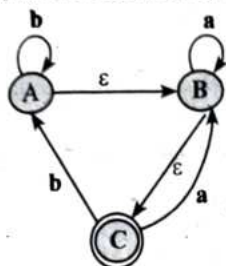


Figure (2): Equivalent DFA Accepting (a|b)abb

Q30. Convert the following NFA with ϵ into equivalent NFA without ϵ .



Answer :

Feb.-22(R18), Q2(a)

Given NFA- ϵ is,

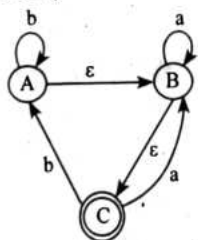


Figure (1): Transition Diagram for NFA- ϵ

ϵ -closure of all states is given as,

$$\epsilon\text{-closure}(A) = \{A, B, C\}$$

$$\epsilon\text{-closure}(B) = \{B, C\}$$

$$\epsilon\text{-closure}(C) = \{C\}$$

Transition function is determined as follows,

$$\delta'(A, \epsilon) = \epsilon\text{-closure}(A) = \{A, B, C\}$$

$$\begin{aligned} \delta'(A, a) &= \epsilon\text{-closure}(\delta(\delta(A, \epsilon), a)) \\ &= \epsilon\text{-closure}(\delta(\{A, B, C\}, a)) \\ &= \epsilon\text{-closure}(\delta(A, a) \cup \delta(B, a) \cup \delta(C, a)) \\ &= \epsilon\text{-closure}(\phi \cup B \cup B) \\ &= \epsilon\text{-closure}(B) \\ &= \{B, C\} \end{aligned}$$

$$\begin{aligned} \delta'(A, b) &= \epsilon\text{-closure}(\delta(\delta(A, \epsilon), b)) \\ &= \epsilon\text{-closure}(\delta(\{A, B, C\}, b)) \\ &= \epsilon\text{-closure}(\delta(A, b) \cup \delta(B, b) \cup \delta(C, b)) \\ &= \epsilon\text{-closure}(A \cup \phi \cup A) \\ &= \epsilon\text{-closure}(A) \\ &= \{A, B, C\} \\ \delta'(B, a) &= \epsilon\text{-closure}(\delta(\delta(B, \epsilon), a)) \\ &= \epsilon\text{-closure}(\delta(\{B, C\}, a)) \\ &= \epsilon\text{-closure}(\delta(B, a) \cup \delta(C, a)) \\ &= \epsilon\text{-closure}(B \cup B) \\ &= \epsilon\text{-closure}(B) \\ &= \{B, C\} \\ \delta'(B, b) &= \epsilon\text{-closure}(\delta(\delta(B, \epsilon), b)) \\ &= \epsilon\text{-closure}(\delta(\{B, C\}, b)) \\ &= \epsilon\text{-closure}(\delta(B, b) \cup \delta(C, b)) \\ &= \epsilon\text{-closure}(\phi \cup A) \\ &= \epsilon\text{-closure}(A) \\ &= \{A, B, C\} \\ \delta'(C, a) &= \epsilon\text{-closure}(\delta(\delta(C, \epsilon), a)) \\ &= \epsilon\text{-closure}(\delta(\{C\}, a)) \\ &= \epsilon\text{-closure}(\delta(C, a)) \\ &= \epsilon\text{-closure}(B) \\ &= \{B, C\} \\ \delta'(C, b) &= \epsilon\text{-closure}(\delta(\delta(C, \epsilon), b)) \\ &= \epsilon\text{-closure}(\delta(\{C\}, b)) \\ &= \epsilon\text{-closure}(\delta(C, b)) \\ &= \epsilon\text{-closure}(A) \\ &= \{A, B, C\} \end{aligned}$$

Transition table for NFA is,

States	Input Symbols	
	a	b
→A	{B, C}	{A, B, C}
B	{B, C}	{A, B, C}
C	{B, C}	{A, B, C}

Table: Transition Table for NFA

Transition diagram of NFA without ϵ -transition is shown in the below figure.

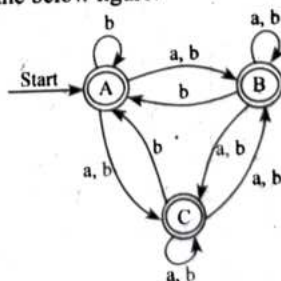


Figure (2): Transition Diagram for NFA

1.3.3 Conversion of NFA to DFA

Q31. Illustrate an example to explain the process used to convert a non-deterministic automata to deterministic automata.

Dec.-19(R16), Q3

OR

Explain the procedure for converting NFA to DFA.

Answer :

Dec.-18(R16), Q3(a)

Procedure to Convert NFA to DFA

Step1: Check for all transitions from initial state on every input symbol in Σ .

Step2: Repeat step (1) for every new state. If a transition on some input symbol resulted in a set of states then it is also considered as a new single state.

Step3: Repeat step (2) until no new states are left.

Step4: The final states of equivalent DFA are all those states which consists of atleast one accepting state of given NFA as its number.

Example

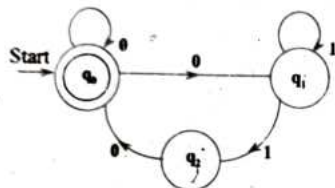


Figure: Transition Diagram for NFA

Given NFA,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_1\}$$

δ is given below,

States	Input Symbols	
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{\phi\}$
q_1	$\{\phi\}$	$\{q_1, q_2\}$
q_2	$\{q_0\}$	$\{\phi\}$

Table: Transition Table for NFA

Let, equivalent DFA be $M' = (Q', \Sigma, \delta', q'_0, F')$

$$\delta(q_0, 0) = \{q_0, q_1\} \quad \dots \text{New state}$$

$$\delta(q_0, 1) = \{\phi\} \quad \dots \text{Empty set}$$

$$\delta(\{q_0, q_1\}, 0) = \{q_0, q_1\} \cup \{\phi\} = \{q_0, q_1\}$$

$$\delta(\{q_0, q_1\}, 1) = \{\phi\} \cup \{q_1, q_2\} = \{q_1, q_2\}$$

$$\delta(\{q_1, q_2\}, 0) = \{\phi\} \cup \{q_0\} = \{q_0\}$$

$$\delta(\{q_1, q_2\}, 1) = \{q_1, q_2\} \cup \{\phi\} = \{q_1, q_2\}$$

$$\therefore Q' = \{q_0, \{q_0, q_1\}, \{q_1, q_2\}\}$$

$$q'_0 = q_0$$

$$F' = \{q_0, \{q_0, q_1\}\}$$

δ' is given below,

States	Input Symbols	
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{\phi\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_0\}$	$\{q_1, q_2\}$

Table: Transition Table for DFA

Transition diagram is,

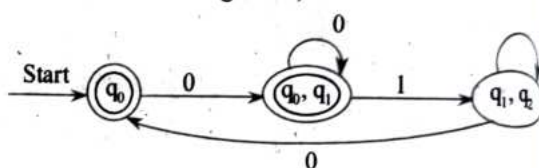


Figure: Transition Diagram for DFA

Example

Let input string = 00110

String Acceptance Using Given NFA

$$\hat{\delta}(q_0, \epsilon) = q_0$$

$$\hat{\delta}(q_0, 0) = \delta(\hat{\delta}(q_0, \epsilon), 0) = \delta(q_0, 0) = \{q_0, q_1\}$$

$$\begin{aligned} \hat{\delta}(q_0, 00) &= \delta(\hat{\delta}(q_0, 0), 0) = \delta(\{q_0, q_1\}, 0) \\ &= \{q_0, q_1\} \cup \{\phi\} = \{q_0, q_1\} \end{aligned}$$

$$\begin{aligned} \hat{\delta}(q_0, 001) &= \delta(\hat{\delta}(q_0, 00), 1) = \delta(\{q_0, q_1\}, 1) \\ &= \{\phi\} \cup \{q_1, q_2\} = \{q_1, q_2\} \end{aligned}$$

$$\begin{aligned} \hat{\delta}(q_0, 0011) &= \delta(\hat{\delta}(q_0, 001), 1) = \delta(\{q_1, q_2\}, 1) \\ &= \{q_1, q_2\} \cup \{\phi\} = \{q_1, q_2\} \end{aligned}$$

$$\begin{aligned} \hat{\delta}(q_0, 00110) &= \delta(\hat{\delta}(q_0, 0011), 0) = \delta(\{q_1, q_2\}, 0) \\ &= \{\phi\} \cup \{q_0\} = \{q_0\} \end{aligned}$$

q_0 is a final state, hence string is accepted by the given NFA.

String Acceptance Using Equivalent DFA

$$\delta(q_0, \epsilon) = q_0$$

$$\delta(q_0, 0) = \delta(\delta(q_0, \epsilon), 0) = \delta(q_0, 0) = \{q_0, q_1\}$$

$$\delta(q_0, 00) = \delta(\delta(q_0, 0), 0) = \delta(\{q_0, q_1\}, 0) = \{q_0, q_1\}$$

$$\delta(q_0, 001) = \delta(\delta(q_0, 00), 1) = \delta(\{q_0, q_1\}, 1) = \{q_1, q_2\}$$

$$\delta(q_0, 0011) = \delta(\delta(q_0, 001), 1) = \delta(\{q_1, q_2\}, 1) = \{q_1, q_2\}$$

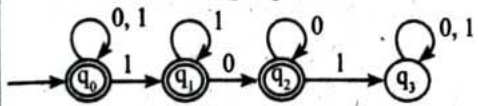
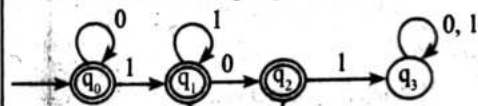
$$\delta(q_0, 00110) = \delta(\delta(q_0, 0011), 0) = \delta(\{q_1, q_2\}, 0) = \{q_0\}$$

q_0 is one of the final state of DFA, hence string is also accepted by DFA.

Q32. Differentiate between NFA and DFA.

Answer :

(Dec.-18(R16), Q2(a) | April-18(R16), Q2(b))

NFA	DFA
- It is possible to define more than one transitions from each state on the same input symbol. - It accepts the input symbol ϵ i.e., empty input symbol. - It takes more time to recognize input string. - It can be smaller than DFA. - It accepts a string if there is some sequence of possible moves that will put the machine in a final state at the string. It rejects string only if there is no possible sequence of moves by which a final state can be reached. Example The NFA for the language of strings that does not contain 101 as substring is given as, 	- It consists of only one transition from each state on any input symbol. - It does not accept empty input symbol. - It is faster than NFA. - It is much bigger than NFA. - It accepts a string if the automaton is in one of its final states when the end of the string is reached. Otherwise it rejects the string. Example The DFA for the language of strings that does not contain 101 as substring is given as, 

Q33. Prove the equivalence of NFA and DFA.

Answer :

Let the non-deterministic finite automata be $M = (Q, \Sigma, \delta, q_0, F)$. An equivalent deterministic finite automata $M' = (Q', \Sigma', \delta', q'_0, F')$ can be constructed as follows,

$$Q' = 2^Q \quad ; \quad \Sigma' = \Sigma$$

$$q'_0 = q_0 \quad ; \quad F' = F$$

- An equivalent DFA accepts the same set of strings defined over an alphabet as that of NFA. Therefore $\Sigma' = \Sigma$.
- The key idea in the construction of DFA is to define single state of DFA that keeps track of all the states M at any given time. Thus, $Q' = 2^Q$. In other words, each state of M' is the subset of the set of states of M .
- A state in Q' is denoted by $[q_1, q_2, \dots, q_i]$ where $q_1, q_2, \dots, q_i$ are individual states in Q .
- An NFA always starts with initial state, so $q'_0 = q_0$.
- An NFA accepts a string w if it is in one of the possible final states at the end of processing w . So final states in M' are all those states of Q' containing at least one final state of M .
- δ' is defined as follows,

Let the machines be at same step in their processing and next input symbol is a . At that step NFA can be in any of the states $q_1, q_2, \dots, q_i$ and can move to any of the states contained in the set $\delta(q_1, a), \delta(q_2, a), \dots, \delta(q_i, a)$ on reading a . If the corresponding DFA is in the state $[q_1, q_2, \dots, q_i]$ then it moves to state,

$$\delta'([q_1, q_2, \dots, q_i], a) = \delta(q_1, a) \cup \delta(q_2, a) \cup \dots \cup \delta(q_i, a).$$

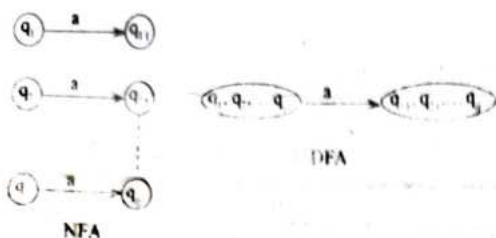
This can also be defined as follows,

$$\delta'([q_1, q_2, \dots, q_i], a) = [r_1, r_2, \dots, r_i]$$

If and only if

$$\delta([q_1, q_2, \dots, q_i], a) = \{r_1, r_2, \dots, r_i\}$$

This could be diagrammatically represented as shown below.



Figure

The constructed DFA accepts exactly the same set of strings as those accepted by the given NFA. At every step of computation the machine M' on any input, enters into a state that corresponds to the set of states that M could be in state at that step.

Hence it is proved that for every NFA there will be an equivalent DFA.

This theorem can be proved by using induction method as follows.

Let us consider 'y' as a length of input string.

$$\delta(q_0, y) = [q_1, q_2, \dots, q_i] \text{ iff}$$

$$\delta(q_0, y) = \{q_1, q_2, \dots, q_i\}$$

Basis Step

If string length is 0 i.e., $|y| = 0$

$$\Rightarrow y = \Sigma \text{ then } q_0' = [q_0]$$

Induction Step

Let, the hypothesis is true for the string of length m or $< m$. If '(ya)' is a string of length $m+1$ then δ' is defined as follows,

$$\delta'(q_0', ya) = \delta'(\delta(q_0, y), a)$$

By using the induction hypothesis,

$$\begin{aligned} \delta'(q_0', ya) &= [r_1, r_2, \dots, r_i] \text{ iff } \delta(q_0, y) \\ &= [r_1, r_2, \dots, r_i] \end{aligned}$$

From the definition of δ'

$$\begin{aligned} \delta'([r_1, r_2, \dots, r_i], a) &= [s_1, s_2, \dots, s_k] \text{ iff } \delta(\{r_1, r_2, \dots, r_i\}, a) \\ &= [s_1, s_2, \dots, s_k] \end{aligned}$$

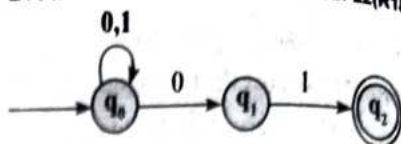
Thus,

$$\delta'(q_0', ya) = [s_1, s_2, \dots, s_k] \text{ iff}$$

$\delta(q_0, ya) = [s_1, s_2, \dots, s_k]$ is proved by making use of induction hypothesis.

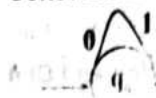
$$\text{So, } L(M) = L(M').$$

Q34. Convert the following NFA into equivalent DFA.
Feb.-22(R18), Q1(b)



OR

Construct DFA for given NFA.



Answer :

Given NFA



The transition table for the given NFA is,

q_i/Σ	0	1
q_0	$\{q_0, q_1\}$	q_0
q_1	ϕ	q_2
q_2	ϕ	ϕ

Let the equivalent DFA be $M' = \langle Q', \Sigma, \delta', q_0', F' \rangle$

$$\delta(q_0, 0) = \{q_0, q_1\} \dots \text{old state}$$

$$\delta(q_0, 1) = q_0 \dots \text{new state}$$

$$\delta(\{q_0, q_1\}, 0) = \{q_0, q_1\} \cup \phi = \{q_0, q_1\}$$

$$\delta(\{q_0, q_1\}, 1) = q_0 \cup q_2 = \{q_0, q_2\}$$

$$\delta(\{q_0, q_2\}, 0) = \{q_0, q_1\} \cup \phi = \{q_0, q_1\}$$

$$\delta(\{q_0, q_2\}, 1) = q_0 \cup \phi = q_0$$

$\therefore$ The δ' is given below,

q'/Σ	0	1
q_0	$\{q_0, q_1\}$	q_0
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	q_0

Transition diagram is,

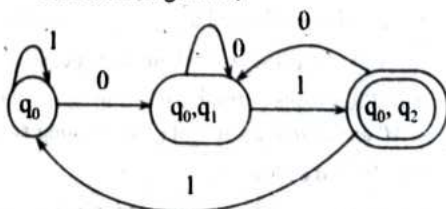
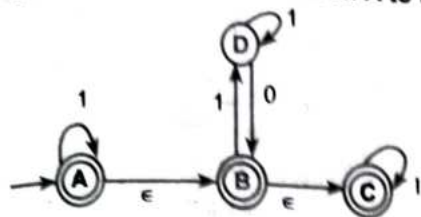


Figure: DFA

Therefore, the above transition diagram is the obtained DFA for the given NFA.

Q35. Convert the following ϵ -NFA to NFA and then convert the resultant NFA to DFA.



Answer :

Given NFA- ϵ is shown below,

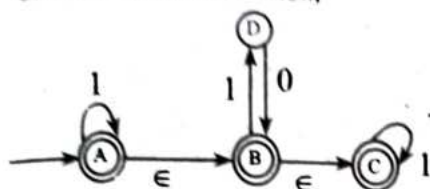


Figure (1): Transition Diagram for NFA- ϵ

ϵ -closure of all states is given as,

$$\epsilon\text{-closure}(A) = \{A, B, C\}$$

$$\epsilon\text{-closure}(B) = \{B, C\}$$

$$\epsilon\text{-closure}(C) = \{C\}$$

$$\epsilon\text{-closure}(D) = \{D\}$$

Transition function is determined as follows,

$$\delta'(A, \epsilon) = \epsilon\text{-closure}(A) = \{A, B, C\}$$

$$\begin{aligned} \delta'(A, 0) &= \epsilon\text{-closure}(\delta(\delta(A, \epsilon), 0)) \\ &= \epsilon\text{-closure}(\delta(\{A, B, C\}, 0)) \\ &= \epsilon\text{-closure}(\delta(A, 0) \cup \delta(B, 0) \cup \delta(C, 0)) \\ &= \epsilon\text{-closure}(\phi \cup \phi \cup \phi) \\ &= \epsilon\text{-closure}(\phi) \\ &= \phi \end{aligned}$$

$$\begin{aligned} \delta'(A, 1) &= \epsilon\text{-closure}(\delta(\delta(A, \epsilon), 1)) \\ &= \epsilon\text{-closure}(\delta(\{A, B, C\}, 1)) \\ &= \epsilon\text{-closure}(\delta(A, 1) \cup \delta(B, 1) \cup \delta(C, 1)) \\ &= \epsilon\text{-closure}(\{A\} \cup \{D\} \cup \{C\}) \\ &= \epsilon\text{-closure}(\{A, D, C\}) \\ &= \{A, B, D, C\} \end{aligned}$$

$$\begin{aligned} \delta'(B, 0) &= \epsilon\text{-closure}(\delta(\delta(B, \epsilon), 0)) \\ &= \epsilon\text{-closure}(\delta(\{B, C\}, 0)) \\ &= \epsilon\text{-closure}(\delta(B, 0) \cup \delta(C, 0)) \\ &= \epsilon\text{-closure}(\phi \cup \phi) \\ &= \epsilon\text{-closure}(\phi) \\ &= \phi \end{aligned}$$

$$\begin{aligned} \delta'(B, 1) &= \epsilon\text{-closure}(\delta(\delta(B, \epsilon), 1)) \\ &= \epsilon\text{-closure}(\delta(\{B, C\}, 1)) \\ &= \epsilon\text{-closure}(\delta(B, 1) \cup \delta(C, 1)) \\ &= \epsilon\text{-closure}(\{D\}, \{C\}) \\ &= \{D, C\} \end{aligned}$$

$$\begin{aligned} \delta'(C, 0) &= \epsilon\text{-closure}(\delta(\delta(C, \epsilon), 0)) \\ &= \epsilon\text{-closure}(\delta(\{C\}, 0)) \\ &= \epsilon\text{-closure}(\delta(C, 0)) \\ &= \epsilon\text{-closure}(\phi) \\ &= \phi \end{aligned}$$

$$\begin{aligned} \delta'(C, 1) &= \epsilon\text{-closure}(\delta(\delta(C, \epsilon), 1)) \\ &= \epsilon\text{-closure}(\delta(\{C\}, 1)) \\ &= \epsilon\text{-closure}(\delta(C, 1)) \\ &= \epsilon\text{-closure}(\{C\}) \\ &= \{C\} \end{aligned}$$

$$\begin{aligned} \delta'(D, 0) &= \epsilon\text{-closure}(\delta(\delta(D, \epsilon), 0)) \\ &= \epsilon\text{-closure}(\delta(\{D\}, 0)) \\ &= \epsilon\text{-closure}(\{B\}) \\ &= \{B, C\} \end{aligned}$$

$$\begin{aligned} \delta'(D, 1) &= \epsilon\text{-closure}(\delta(\delta(D, \epsilon), 1)) \\ &= \epsilon\text{-closure}(\delta(\{D\}, 1)) \\ &= \epsilon\text{-closure}(\{D\}) \\ &= \{D\} \end{aligned}$$

Transition table for NFA is,

States	Input Symbols	
	0	1
$\rightarrow A$	-	{A, B, D, C}
B	-	{D, C}
C	-	{C}
D	{B, C}	{D}

Table (1): Transition Table for NFA

Transition diagram of NFA without ϵ -transitions is shown in the below figure,

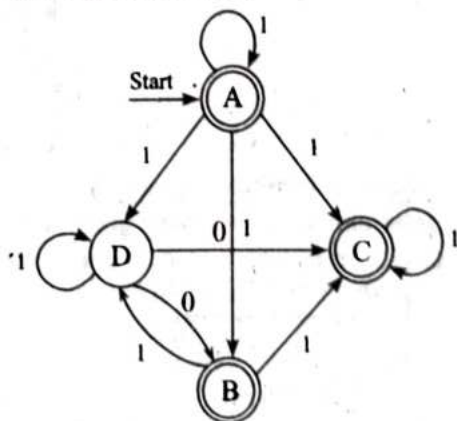


Figure (2): Transition Diagram for NFA

Let equivalent DFA be, $M' = (Q', \Sigma', \delta', q_0', F')$

$$\delta(A, 0) = \{\phi\} \quad \dots \text{Empty State}$$

$$\delta(A, 1) = \{A, D, C\} \quad \dots \text{New State}$$

$$\delta(\{A, B, D, C\}, 0) = \{\phi\} \cup \{\phi\} \cup \{B, C\} \cup \{\phi\} \\ = \{B, C\} \quad \dots \text{New State}$$

$$\delta(\{A, B, D, C\}, 1) = \{A, B, D, C\} \cup \{D, C\} \cup \\ \{D\} \cup \{C\} \\ = \{A, B, D, C\} \quad \dots \text{Old State}$$

$$\delta(\{B, C\}, 0) = \{\phi\} \cup \{\phi\} \\ = \{\phi\} \quad \dots \text{Empty State}$$

$$\delta(\{B, C\}, 1) = \{D, C\} \cup \{C\} \\ = \{D, C\} \quad \dots \text{New State}$$

$$\delta(\{D, C\}, 0) = \{B, C\} \cup \{\phi\} \\ = \{B, C\} \quad \dots \text{Old State}$$

$$\delta(\{D, C\}, 1) = \{C\} \cup \{D\} \\ = \{D, C\} \quad \dots \text{Old State}$$

$$Q' = \{A, \{A, D, C\}, \{B, C\}, \{D, C\}\}$$

$$q_0' = A$$

$$F' = \{\{A, B, D, C\}, \{B, C\}, \{D, C\}\}$$

δ' is given below,

States	Input Symbols	
	0	1
$\rightarrow A$	$\{\phi\}$	$\{A, D, C\}$
$\{A, B, D, C\}$	$\{B, C\}$	$\{A, B, D, C\}$
$\{B, C\}$	$\{\phi\}$	$\{D, C\}$
$\{D, C\}$	$\{B, C\}$	$\{D, C\}$

Table (2): Transition Table for DFA

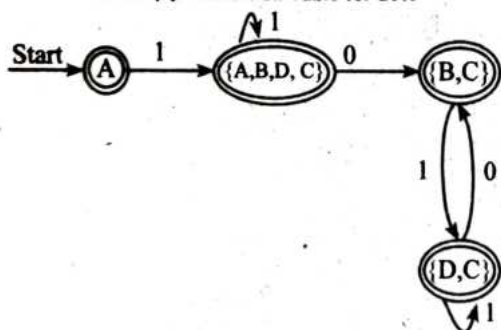


Figure (3): Transition Diagram for DFA

Q36. Determine the DFA from a given NFA
 $M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_1\})$ with the state table diagram for δ given below.

δ	a	b
q_0	$\{q_0, q_1\}$	$\{q_1\}$
q_1	ϕ	$\{q_0, q_1\}$

Answer :

Given,

$$NFA(M) = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_1\})$$

Where,

q_0 is the initial state

q_1 is the final state.

The equivalent DFA is,

$$\delta(q_0, a) = \{q_0, q_1\} \quad \dots \text{New state}$$

$$\delta(q_0, b) = \{q_1\} \quad \dots \text{New state}$$

$$\delta(\{q_0, q_1\}, a) = \{q_0, q_1\} \cup \{\phi\} \\ = \{q_0, q_1\} \quad \dots \text{Old state}$$

$$\delta(\{q_0, q_1\}, b) = \{q_1\} \cup \{q_0, q_1\} \\ = \{q_0, q_1\} \quad \dots \text{Old state}$$

$$\delta(q_1, a) = \phi \quad \dots \text{Empty state}$$

$$\delta(q_1, b) = \{q_0, q_1\} \quad \dots \text{Old state.}$$

The transition table for DFA is,

States	Input Symbols	
	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_1\}$
$\{q_1\}$	ϕ	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$

Table: Transition Table for DFA

The transition diagram for DFA is,

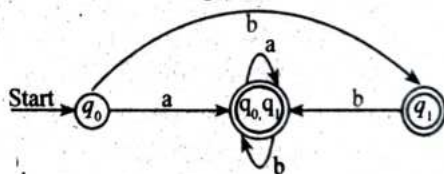


Figure (1): Transition Diagram for DFA

Here, $\{q_1\}$ and $\{q_0, q_1\}$ both are final states, as $\{q_0, q_1\}$ consists of final state $\{q_1\}$

Let us consider $P = \{q_0\}$

$$Q = \{q_0, q_1\}$$

$$R = \{q_1\}$$

So, the new transition diagram for the equivalent DFA is shown below,

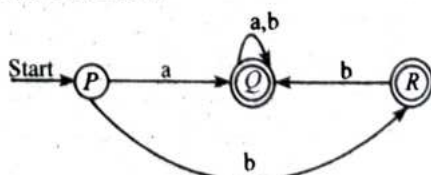


Figure (2): Transition Diagram for the Equivalent DFA

Q37. Convert the following NFA into equivalent DFA.

δ	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_1
q_1	ϕ	$\{q_0, q_1\}$

Answer :

March-21(2018), Q1(h)

Given,

$$\text{NFA } (M) = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

Where,

q_0 is the initial state

q_1 is the final state.

The equivalent DFA is,

$$\delta(q_0, 0) = \{q_0, q_1\} \quad \dots \text{New state}$$

$$\delta(q_0, 1) = \{q_1\} \quad \dots \text{New state}$$

$$\begin{aligned} \delta(\{q_0, q_1\}, 0) &= \{q_0, q_1\} \cup \{\phi\} \\ &= \{q_0, q_1\} \quad \dots \text{Old state} \end{aligned}$$

$$\begin{aligned} \delta(\{q_0, q_1\}, 1) &= \{q_1\} \cup \{q_0, q_1\} \\ &= \{q_0, q_1\} \quad \dots \text{Old state} \end{aligned}$$

$$\delta(q_1, 0) = \phi \quad \dots \text{Empty state}$$

$$\delta(q_1, 1) = \{q_0, q_1\} \quad \dots \text{Old state}$$

The transition table for DFA is,

States	Input Symbols	
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_1
q_1	ϕ	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$

Table: Transition Table for DFA

The transition diagram for DFA is,

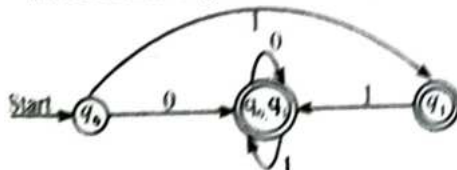


Figure (1): Transition Diagram for DFA

Here, $\{q_1\}$ and $\{q_0, q_1\}$ both are final states, as $\{q_0, q_1\}$ consists of final state $\{q_1\}$

Let us consider $P = \{q_0\}$

$$Q = \{q_0, q_1\}$$

$$R = \{q_1\}$$

So, the new transition diagram for the equivalent DFA is shown below.



Figure (2): Transition Diagram for the Equivalent DFA

Q38. Eliminate the ϵ -transactions of the following NFA.

State	a	b	c
Q_0	Q_0	Q_1	Q_2
Q_1	Q_1	Q_2	Q_3
Q_2	Q_2	Q_3	
Q_3	Q_0	-	

Answer :

Sep-21(2018), Q2(h)

Given transition table (b) is,

State	a	b	c
Q_0	Q_0	Q_1	Q_2
Q_1	Q_1	Q_2	Q_3
Q_2	Q_2	Q_3	-
Q_3	Q_0	-	-

In the given transition table initial state is not mentioned. So, consider ' Q_0 ' as initial state.

The transition diagram for given transition table can be drawn as follows,

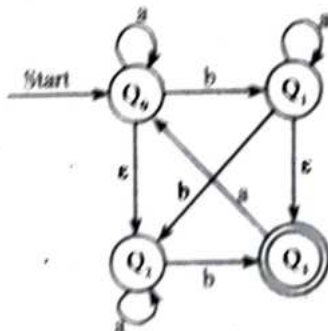


Figure (1): Transition Diagram for NFA- ϵ

ϵ -closure of all states is given as,

$$\epsilon\text{-closure}(Q_0) = \{Q_0, Q_2\}$$

$$\epsilon\text{-closure}(Q_1) = \{Q_1, Q_3\}$$

$$\epsilon\text{-closure}(Q_2) = \{Q_2\}$$

$$\epsilon\text{-closure}(Q_3) = \{Q_3\}$$

Transition function is determined as follows,

$$\delta'(Q_0, \epsilon) = \epsilon\text{-closure}(Q_0)$$

$$= \{Q_0, Q_2\}$$

$$\delta'(Q_0, a) = \epsilon\text{-closure}(\delta(\delta'(Q_0, \epsilon), a))$$

$$= \epsilon\text{-closure}(\delta(\{Q_0, Q_2\}, a))$$

$$= \epsilon\text{-closure}(\delta(Q_0, a) \cup \delta(Q_2, a))$$

$$= \epsilon\text{-closure}(\{Q_0\} \cup \{Q_2\})$$

$$= \epsilon\text{-closure}(\{Q_0, Q_2\})$$

$$= \{Q_0, Q_2\}$$

$$\delta'(Q_0, b) = \epsilon\text{-closure}(\delta(\delta'(Q_0, \epsilon), b))$$

$$= \epsilon\text{-closure}(\delta(\{Q_0, Q_2\}, b))$$

$$= \epsilon\text{-closure}(\delta(Q_0, b) \cup \delta(Q_2, b))$$

$$= \epsilon\text{-closure}(\{Q_1\} \cup \{Q_3\})$$

$$= \epsilon\text{-closure}(\{Q_1, Q_3\})$$

$$= \{Q_1, Q_3\}$$

$$\delta'(Q_1, a) = \epsilon\text{-closure}(\delta(\delta'(Q_1, \epsilon), a))$$

$$= \epsilon\text{-closure}(\delta(\{Q_1, Q_3\}, a))$$

$$= \epsilon\text{-closure}(\delta(Q_1, a) \cup \delta(Q_3, a))$$

$$= \epsilon\text{-closure}(\{Q_1\} \cup \{Q_0\})$$

$$= \epsilon\text{-closure}(\{Q_0, Q_1\})$$

$$= \{Q_0, Q_1, Q_2, Q_3\}$$

$$\delta'(Q_1, b) = \epsilon\text{-closure}(\delta(\delta'(Q_1, \epsilon), b))$$

$$= \epsilon\text{-closure}(\delta(\{Q_1, Q_3\}, b))$$

$$= \epsilon\text{-closure}(\delta(Q_1, b) \cup \delta(Q_3, b))$$

$$= \epsilon\text{-closure}(\{Q_2\} \cup \{\phi\})$$

$$= \epsilon\text{-closure}(\{Q_2\})$$

$$= \{Q_2\}$$

$$\delta'(Q_2, a) = \epsilon\text{-closure}(\delta(\delta'(Q_2, \epsilon), a))$$

$$= \epsilon\text{-closure}(\delta(\{Q_2\}, a))$$

$$= \epsilon\text{-closure}(Q_2)$$

$$= \{Q_2\}$$

$$\delta'(Q_2, b) = \epsilon\text{-closure}(\delta(\delta'(Q_2, \epsilon), b))$$

$$= \epsilon\text{-closure}(\delta(\{Q_2\}, b))$$

$$= \epsilon\text{-closure}(Q_3)$$

$$= \{Q_3\}$$

$$\delta'(Q_3, a) = \epsilon\text{-closure}(\delta(\delta'(Q_3, \epsilon), a))$$

$$= \epsilon\text{-closure}(\delta(\{Q_3\}, a))$$

$$= \epsilon\text{-closure}(Q_0)$$

$$= \{Q_0, Q_2\}$$

$$\delta'(Q_3, b) = \epsilon\text{-closure}(\delta(\delta'(Q_3, \epsilon), b))$$

$$= \epsilon\text{-closure}(\delta(\{Q_3\}, b))$$

$$= \epsilon\text{-closure}(\phi)$$

$$= \phi$$

Transition table for NFA is,

States	Input Symbols	
	a	b
$\rightarrow Q_0$	$\{Q_0, Q_2\}$	$\{Q_1, Q_3\}$
Q_1	$\{Q_0, Q_1, Q_2, Q_3\}$	$\{Q_2\}$
Q_2	$\{Q_2\}$	$\{Q_3\}$
$\odot Q_3$	$\{Q_0, Q_2\}$	ϕ

Table: Transition Table for NFA

Transition diagram for NFA without ϵ -transitions is shown in the below figure.

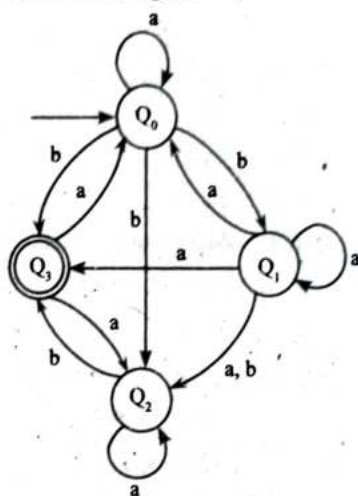


Figure: Transition Diagram for NFA

VERY SHORT QUESTIONS WITH SOLUTIONS (VSQS)

Q1. What is finite automata?

Answer :
A finite automata or finite state machine is a mathematical model that takes string (w) as its input and check whether it belongs to the given language or not. A finite state machine is mainly used in designing text editors, language processors and lexical analyzers.

A finite automata (M) is designed using the five tuples such as,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q2. Define transition diagram.

Dec.-17(R13), Q1(a)

Answer :
A transition diagram describes the behavior of a finite automata. It consists of a collection of nodes or circles called states and directed line segments between each state called node edges.

Q3. Define transition table.

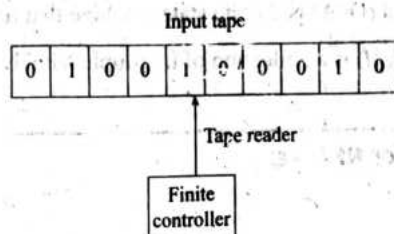
May-17(R13), Q1(a)

Answer :
Transition table is the tabular representation of a given finite automata. It consists of rows and columns, wherein rows correspond to the input state and the column corresponds to the input symbol of given input state.

Q4. What is the mathematical model of finite automata?

Dec.-19(R16), Q1(b)

Answer :
The mathematical model of finite automata is shown in figure below,



Q5. Define the central concepts of automata theory.

Dec.-18(R16), Q1(a)

Answer :
The central concepts of automata theory are,
(a) **Alphabets:** An alphabet or character is any non empty finite set of symbols.
(b) **Strings:** A finite sequence of symbol drawn from alphabet is known as string.
(c) **Language:** The language denoted by L is a set of string over alphabet Σ .
(d) **Problem:** A problem can be defined as a question which is useful to determine whether a string belongs to specific language.

Q6. What is a string? Write about concatenation of two strings.

Jan./Feb.-23(R18), Q1(a)

Answer :
String

A finite sequence of symbols drawn from an alphabet is known as string ' S '. For example, if the alphabet, $\Sigma = \{0, 1\}$ then string S drawn from this alphabet is 001100.

Concatenation of Two Strings

Concatenation of two languages L_1 and L_2 , denoted as $L_1 L_2$ or $L_1 \circ L_2$, contains the strings from L_1 followed by strings from L_2 .

$$L_1 L_2 = \{st \mid w_1 \text{ in } L_1 \text{ and } w_2 \text{ is in } L_2\}$$

Q7. Define Non-deterministic Finite Automata.

Answer :

A Non-deterministic Finite Automata (M) is a collection of five-tuples, namely,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q8. Define Kleene closure and positive closure.

Answer :

Kleene Closure

Kleene star of a language L , denoted as L^* contains all the strings obtained by concatenating zero or more strings from L .

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Where, $L^i = L^{i-1} L$

Positive Closure

Positive Closure of a language L , denoted as L^+ contains all the strings obtained by concatenating one or more strings from L .

$$L^+ = \bigcup_{i=1}^{\infty} L^i$$

Q9. Define DFA.

Answer :

A Deterministic Finite Automata (DFA) is a finite state machine that recognizes a given language.

A deterministic finite automata (M) is a collection of five-tuple namely,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q10. Write the design strategy for NFA - ϵ .

Answer :

Design Strategy of NFA- ϵ

The following are the steps followed to design a strategy for NFA- ϵ .

1. Firstly, the NFA- ϵ is converted to DFA.
2. The number of states of DFA is minimized.
3. A transition table is used to represent the minimized DFA.
4. The generated transition table is then used to write code for the NFA- ϵ .
5. The code is finally tested.

FREQUENTLY ASKED QUESTIONS & IMPORTANT QUESTIONS

Q1. Define Finite Automaton. Explain about the model of Finite Automaton.

Answer : (Important Question | Jan./Feb.-23(R18), Q2(a))

For answer refer Unit-I, Page No. 2, Q1.

Q2. Briefly explain the structural representations of finite automata.

Answer : Important Question

For answer refer Unit-I, Page No. 3, Q3.

Q3. Describe in brief about applications of Finite Automata.

Answer : (Jan./Feb.-23(R18), Q3(a) | Dec.-18(R16), Q1(b) | April-18(R16), Q1(b))

For answer refer Unit-I, Page No. 4, Q5.

Q4. Design NFA to accept strings with a's and b's such that the string end with bb.

Answer : (Important Question | May-13(R09), Q1(a))

For answer refer Unit-I, Page No. 8, Q11.

Q5. How NFA is used for searching a text?

Answer : Important Question

For answer refer Unit-I, Page No. 9, Q13.

Q6. Briefly discuss about Finite Automata with Epsilon-transitions.

Answer : (Dec.-18(R16), Q3(b) | Dec.-17(R13), Q1(b) | May-17(R13), Q2(a))

For answer refer Unit-I, Page No. 9, Q14.

Q7. Explain the uses of ϵ -transitions with an example.

Answer : Important Question

For answer refer Unit-I, Page No. 10, Q16.

Q8. Define Deterministic Finite Automata (DFA). Explain about the significance of DFA.

Answer : Important Question

For answer refer Unit-I, Page No. 11, Q18.

Q9. What is extended transition function of DFA? Discuss about the languages accepted by a DFA.

Answer : Important Question

For answer refer Unit-I, Page No. 12, Q20.

Q10. Construct a DFA to accept the binary strings divisible by 5.

Answer : (Sep.-21(R18), Q2(a) | Dec.-19(R16), Q2)

For answer refer Unit-I, Page No. 12, Q21

Q11. Construct a DFA accepting the set of strings ending with 00.

Answer : (Important Question | April-18(R16), Q3(b))

For answer refer Unit-I, Page No. 13, Q23.

Q12. Design a FA for the Language which accepts odd number of 0's and odd number of 1's over input alphabet $\Sigma = \{0, 1\}$.

Answer :

(Important Question | Feb.-22(R18), Q1(a))

For answer refer Unit-I, Page No. 15, Q26.

Q13. Construct NFA with ϵ which accepts a language consisting all strings of any number of 0's followed by any number of 1's followed by any number of 2's. And also convert into NFA without ϵ -transitions.

Answer :

(Important Question | May-19(R16), Q2)

For answer refer Unit-I, Page No. 16, Q28.

Q14. Illustrate an example to explain the process used to convert a non-deterministic automata to deterministic automata.

Answer :

(Dec.-19(R16), Q3 | Dec.-18(R16), Q3(a))

For answer refer Unit-I, Page No. 20, Q31.

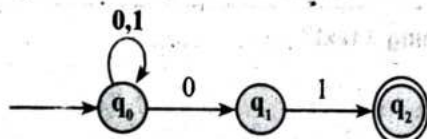
Q15. Differentiate between NFA and DFA.

Answer :

(Dec.-18(R16), Q2(a) | April-18(R16), Q2(b))

For answer refer Unit-I, Page No. 21, Q32.

Q16. Convert the following NFA into equivalent DFA.



Answer :

(Feb.-22(R18), Q1(b) | May-15(R13), Q2(b))

For answer refer Unit-I, Page No. 22, Q34.

Q17. Convert the following NFA into equivalent DFA.

δ	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_1
q_1	ϕ	$\{q_0, q_1\}$

Answer :

March-21(R18), Q1(b)

For answer refer Unit-I, Page No. 25, Q37.